

Bibliographie annotée

Sayan Suos – 29 avril 2026

Sujet du stage : Étude d’algorithmes d’apprentissage par renforcement pour l’optimisation en temps du transport des bagages par des véhicules guidés automatisés (AGV) en environnement aéroportuaire.

Objectif du stage : L’objectif est de minimiser le temps total de transport en apprenant aux AGV à contrôler leur vitesse, afin d’éviter les collisions, et d’accomplir leur mission le plus rapidement possible. On considère un cadre décentralisé, sans méta-agent et sans communication explicite entre les AGV.

Références

- [1] T. Maisonhaute, F. Michel, and J-C. Soulié. État de l’art sur les approches en apprentissage par renforcement multi-agent. In *Journées Francophones sur les Systèmes Multi-Agents (JFSMA)*, pages 99–108, 2024. 3
- [2] R. Singh, J. Ren, and X. Lin. A review of deep reinforcement learning algorithms for mobile robot path planning. *Vehicles*, 5(4) :1423–1451, 2023. 5
- [3] A. Amini. MIT 6.S191 : Reinforcement learning, 2025. Vidéo YouTube, consultée le 1er avril 2026. 9
- [4] R. Reijnen, Y. Zhang, W. P. M. Nuijten, C. Senaras, and M. Goldak. Combining deep reinforcement learning with search heuristics for solving multi-agent path finding in segment-based layouts. In *IEEE Symposium Series on Computational Intelligence (SSCI)*, pages 2647–2654, 2020. 11, 12
- [5] B. Wang, Z. Liu, Q. Li, and A. Prorok. Mobile robot path planning in dynamic environments through globally guided reinforcement learning. *IEEE Robotics and Automation Letters*, 5(4) :6932–6939, 2020. 14
- [6] J. Choi, G. Lee, and C. Lee. Reinforcement learning-based dynamic obstacle avoidance and integration of path planning. *International Journal of Intelligent Robotics and Applications*, 14(5) :663–677, 2021. 17, 18
- [7] Z. Huang, Z. Ren, T. Chen, S. Cai, and C. Xu. Autonomous navigation and collision avoidance for agv in dynamic environments : An enhanced deep reinforcement learning approach with composite rewards and dynamic update mechanisms. *IET Cyber-Systems and Robotics*, 2024. 21, 22
- [8] J. Liang, U. Patel, A. J. Sathyamoorthy, and D. Manocha. Realtime collision avoidance for mobile robots in dense crowds using implicit multi-sensor fusion and deep reinforcement learning. *arXiv preprint arXiv :2004.03089*, 2020. 24, 25
- [9] P. Long, T. Fan, X. Liao, W. Liu, H. Zhang, and J. Pan. Towards optimally decentralized multi-robot collision avoidance via deep reinforcement learning. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2018. 26, 46, 47
- [10] R. Han, S. Chen, S. Wang, Z. Zhang, R. Gao, Q. Hao, and J. Pan. Reinforcement learned distributed multi-robot navigation with reciprocal velocity obstacle shaped rewards. *IEEE Robotics and Automation Letters*, 7(3) :5896–5903, 2022. 28, 30
- [11] Y. Fan Chen, M. Liu, M. Everett, and J. P. How. Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning. In *2017 IEEE International Conference on Robotics and Automation (ICRA)*, pages 285–292, 2017. 32
- [12] G. Chen, S. Yao, J. Ma, L. Pan, Y. Chen, P. Xu, J. Ji, and X. Chen. Distributed non-communicating multi-robot collision avoidance via map-based deep reinforcement learning. *Sensors*, 20(17) :4836, 2020. 35, 36

- [13] E. Escudie, L. Matignon, and J. Saraydaryan. Attention graph for multi-robot social navigation with deep reinforcement learning. *arXiv preprint arXiv :2401.17914*, 2024. 39
- [14] M. De Ryck, M. Versteyhe, and F. Debrouwere. Automated guided vehicle systems, state-of-the-art control algorithms and techniques. *Journal of Manufacturing Systems*, 54 :152–173, 2020. 42, 43
- [15] S. V. Albrecht. SESSION 3 | Multi-Agent Reinforcement Learning : Foundations and Modern Approaches | IAAA-CSIC Course, 2024. Vidéo YouTube, consultée le 20 avril 2026. 50
- [16] X. Ye, Z. Deng, Y. Shi, and W. Shen. Toward energy-efficient routing of multiple agvs with multi-agent reinforcement learning. *Sensors*, 23(12) :5615, 2023. 51

[1] État de l’art sur les approches en apprentissage par renforcement multi-agent, Maisonhaute & al. (2024)

SUJET

L’article présente l’état de l’art sur les approches d’apprentissage par renforcement multi-agent, avec pour objectif d’identifier les meilleures solutions selon le contexte.

RÉSUMÉ

En apprentissage par renforcement (RL), un agent prend des décisions selon l’état de son environnement, orientées par des récompenses, afin d’accomplir un objectif. De fait, un algorithme de RL optimise la politique d’un agent pour maximiser ses récompenses. Pour cela, on se place très souvent dans le cadre des jeux de Markov, selon l’hypothèse que la récompense perçue par un agent ne dépend que de l’état courant (propriété de Markov).

On définit alors 3 fonctions : R_t qui correspond au gain perçu depuis l’instant t jusqu’à la fin de l’épisode, V_π qui correspond au gain espéré pour l’état s sachant que l’agent suit la politique π_θ , et Q_π qui correspond au gain espéré pour l’état s en réalisant l’action a_t , sachant que l’agent suit la politique π_θ .

$$R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}, \quad V_\pi(s) = \mathbb{E}_\pi [R_0 \mid s_0 = s], \quad Q_\pi(s, a) = \mathbb{E}_\pi [R_0 \mid s_0 = s, a_0 = a],$$

où γ est le facteur de discount et r_t est la récompense à l’instant t .

Pour l’apprentissage de la politique de l’agent, on peut utiliser (i) une politique qui évolue selon l’algorithme et les interactions (on-policy methods), ou (ii) une politique décisionnelle et une politique cible (off-policy methods). Certains algorithmes définissent leur politique autour des fonctions d’action-valeur ou des fonctions valeur (value-based algorithms), alors que d’autres mettent à jour la politique de l’agent (policy-based algorithms).

Dans le cadre du multi-agent reinforcement learning (MARL), plusieurs agents interagissent et influencent l’environnement et les récompenses du groupe. Dans certaines situations, on peut retrouver un équilibre de Nash : chaque agent applique la meilleure politique possible compte-tenu des choix des autres. Cet équilibre de Nash est de type différent, selon si les stratégies sont pures ou mixtes.

Plus particulièrement un équilibre de Nash existe toujours si : (i) les stratégies mixtes sont autorisées, (ii) le nombre de joueurs est fini et (iii) le nombre d’actions de chaque joueur est fini.

Les auteurs mentionnent 5 problèmes engendrés par le cas multi-agents :

1. **Non-stationnarité.** Les agents apprennent et modifient leur action en parallèle, on perd alors la propriété de Markov.
2. **Définition de l’objectif.** Les deux grands objectifs en MARL sont la recherche de stabilité, i.e. atteindre une politique stationnaire, et la recherche d’adaptabilité, i.e., adapter sa stratégie à celle des autres. Ces deux éléments peuvent être satisfaits par la convergence vers un équilibre de Nash.
3. **Connaissance partielle de l’environnement.** Les agents choisissent leur action à partir d’observations partielles de l’environnement, ils sont incapables de distinguer deux états qui semblent identiques.
4. **Scalabilité.** L’espace de l’ensemble des actions s’élargit avec le nombre d’agents, ce qui rend les algorithmes coûteux.
5. **Hétérogénéité.**

Les auteurs présentent ensuite plusieurs analyses qui permettent de traiter les problèmes de MARL en considérant de manière isolée ces problématiques.

- **Independent Learning (IL).** Les agents ne prennent pas en compte le comportement des autres, ce qui permet de faire abstraction du problème de non-stationnarité. Cet algorithme est surtout à comparer à d'autres, avec la conclusion que la coopération entre agents est souvent préférable.
- **Fictitious Play (FP).** Les agents considèrent que les autres utilisent une stratégie mixte stationnaire, ce qui permet de faire abstraction de l'évolution de leur comportement. L'agent choisit une action en fonction de la politique calculée à un instant donnée pour les autres agents.
- **Deep MARL (MADRL).** Cette méthode est basée sur des réseaux de neurones pour faire du RL.
- **Centralized Training Decentralized Execution (CTDE).** La simulation est séparée en deux phases : l'apprentissage et l'exécution, les agents utilisent durant l'apprentissage des informations auxquelles ils n'auront plus accès durant l'exécution. Cela est particulièrement pertinent en cas d'observabilité partielle, lorsqu'il y a un très grand nombre d'agents.
- **Networked Agents.** Les agents communiquent entre eux durant l'apprentissage pour pallier l'observabilité partielle.
- **Two-Timescale analysis.** On définit deux vitesses de mise à jour différentes pour deux processus, ce qui permet au processus rapide d'être toujours adapté au processus lent, et pallie le problème de non-stationnarité.
- **Mean Field (MF MARL).** Cette méthode est utilisée dans le cas où il y a plus de deux agents, on considère le problème "moi vs les autres" en utilisant un champ moyen. On peut aussi considérer un voisinage plutôt que tous les autres. Cette méthode est surtout efficace lorsque les agents sont homogènes.

CONCLUSION

Dans le cadre de notre étude, on s'intéresse au problème de collision avoidance entre plusieurs AGV, en supposant l'absence de communication explicite entre les agents et une prise de décision basée sur des observations locales (pas de méta-agent).

Dans ce contexte, les approches les plus pertinentes reposent sur du MADRL, qui permet de traiter des environnements complexes et continus. En particulier, le CTDE semble adapté car il permet de conserver les informations globales durant l'apprentissage, tout en conservant une exécution décentralisée compatible avec l'observation partielle. En fonction du nombre d'AGV, on pourrait également envisager des méthodes de MF MARL afin d'améliorer la scalabilité des méthodes.

Au contraire, les méthodes reposant sur la communication des agents, telles que les Networked Agents, ne correspondent pas aux hypothèses retenues. De même, les approches classiques de IL ou de FP paraissent moins adaptées.

[2] A review of deep reinforcement learning algorithms for mobile robot path planning, Singh & al. (2023)

SUJET

Ce papier traite du problème de path-planning pour les robots mobiles autonomes, en proposant des méthodes de deep reinforcement learning (DRL).

RÉSUMÉ

Les robots mobiles autonomes peuvent être utilisés pour différentes tâches, telles que le nettoyage, la livraison, ou la surveillance. Pour mener à bien ces missions, ils disposent d'une observation sur leur environnement, collectée par des capteurs, telles que des caméras.

L'objectif est de déterminer un chemin optimal, qui permet au robot d'atteindre une position cible, sans entré en collision. Avec l'essor du deep learning, de nouvelles méthodes sont apparues pour résoudre ce type de problème.

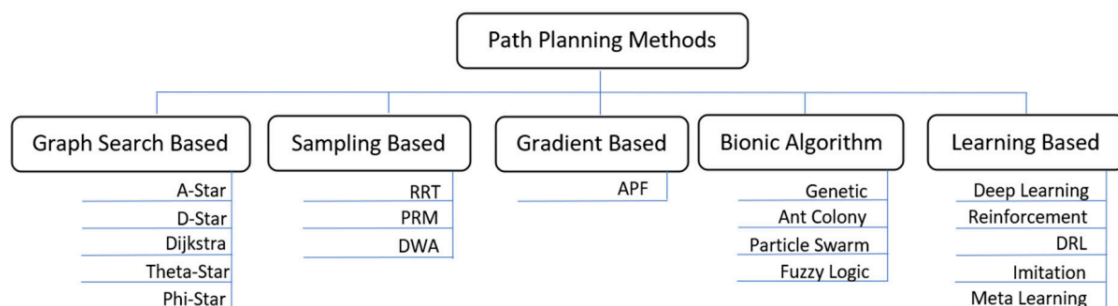


FIGURE 1 – Résumé des méthodes de path planning. Source : [2]

Deep learning (DL). On utilise un réseau de neurones pour apprendre à reconnaître des patterns, ou pour approximer des fonctions et classifier des images. On peut donc l'utiliser de plusieurs manières pour résoudre le problème de path-planning.

Imitation learning (IL). On utilise un réseau de neurones profond pour apprendre la relation entre un environnement et des décisions prises par un conducteur expert. De cette manière, les démonstrations sont vues comme des labels par le modèle. Cette méthode fonctionne assez bien avec une grande base de données d'entraînement mais peut facilement engendrer du sur-apprentissage ou une difficulté à généraliser à des scénarios qui n'ont pas été vus durant l'entraînement. Plus particulièrement, le conducteur expert ne subit pas de collisions en pratique, donc l'agent n'a peut-être pas appris à réagir correctement une fois qu'il est en voie de collision.

Reinforcement learning (RL). Un agent apprend le comportement à suivre à partir d'interactions avec son environnement et de récompenses reçues. Les modèles d'apprentissage par renforcement peuvent être model-based ou model-free, selon si les algorithmes doivent apprendre ou utiliser les probabilités de transition ou non.

Deep reinforcement learning (DRL). On combine la capacité de prise de décision de l'apprentissage par renforcement et celle d'identification de caractéristiques des réseaux de neurones pour résoudre le problème de path-planning. De cette manière, le problème de la grande dimension peut être résolu en approxinant la fonction Q optimale.

La première méthode a combiné un CNN et un algorithme de Q-learning et a été appliquée à plusieurs cas (AlphaGo). Le DRL a connu un succès pour résoudre le problème de path-planning et d'évitement de collision.

Globalement, le réseau de neurones prend en entrée des images et une position cible, retourne un déplacement à faire, et reçoit une récompense en fonction : l'objectif est de choisir les meilleures actions qui maximisent la récompense, soit en améliorant les valeurs de la fonction Q , ou bien en améliorant directement la politique π .

- **Value-based methods.** On utilise un réseau de neurones profond pour approximer les valeurs de la fonction Q et on modifie les valeurs de Q ou de V par temporal difference learning (TD) ou Q-learning. Le Q-learning consiste à continuellement optimiser la stratégie à partir des informations reçues par l'agent sur l'environnement.

Deep Q Network (DQN). L'agent choisit ses actions selon une politique ϵ -gloutonne, dont la valeur change au cours de l'entraînement, de sorte que l'agent explore au début et exploite à la fin. C'est une off-policy method : la fonction Q est apprise indépendamment des décisions de l'agent. Durant l'entraînement, deux réseaux de neurones sont utilisés, et disposent de paramètres différents : (i) un prediction network qui prend en entrée l'état s et prédit $Q_{pred}(s, a)$, et (ii) un target network qui calcule la cible y ,

$$y = r + \gamma \max_{a'} Q_{target}(s', a').$$

Le prediction network est continuellement mis à jour, tandis que le target network est mis à jour périodiquement. Pour cela, les transitions sont gardées en mémoire dans un replay buffer (s, a, r, s') . Le modèle est optimisée en minimisant l'erreur quadratique entre la cible et la valeur prédite (MSE), par descente de gradient. Cependant : cet algorithme peut facilement surestimer les valeurs de la Q-fonction et la corrélation entre les échantillons est toujours assez haute.

Double DQN (DDQN). Pour pallier ce problème, on sépare la sélection de l'action optimale et son évaluation entre les deux réseaux de neurones. Le prediction network choisit la meilleure action a^* et le target network va évaluer sa valeur et calculer la cible y ,

$$a^* = \arg \max_a Q_{pred}(s', a), \quad y = r + \gamma \max_{a'} Q_{target}(s', a^*).$$

Dueling DQN (D3QN). L'algorithme décompose la valeur de Q en une valeur-état V et une valeur-avantage A , qui mesure à quel point une action a est avantageuse dans l'état s . Cela permet de différencier l'action actuelle et la performance moyenne. De cette manière, le modèle peut apprendre quel état est ou n'est pas avantageux, sans avoir à apprendre les effets de chaque action sur chaque état. On sépare le réseau en deux streams : l'une qui estime la valeur de l'état $V(s)$, et l'autre qui estime l'avantage de chaque action $A(s, a)$, puis on les combine (le plus souvent, en soustrayant la moyenne),

$$Q(s, a) = V(s) + A(s, a) - \frac{1}{|A|} \sum_{s'} A(s, a').$$

- **Policy-based methods.** La méthode de policy gradient a pour objectif de trouver la meilleure politique $\pi_\theta(a | s)$. Pour cela, on souhaite maximiser la fonction objective, qui correspond à l'espérance de la récompense cumulée :

$$J(\theta) = \mathbb{E}_{\pi_\theta} [R_0] \quad \text{où} \quad R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}.$$

On réécrit J et on obtient son gradient :

$$J(\theta) = \mathbb{E}_{\pi_\theta} [Q(s, a)] = \sum_a \pi_\theta(a | s) Q(s, a), \quad \nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a | s) Q(s, a)].$$

Durant l'entraînement, on exécute la politique π_θ et on stocke les transitions (s, a, r, s') jusqu'à la fin de l'épisode. On en déduit le gain R_t pour chaque étape, et on met à jour les paramètres via une descente de gradient stochastique :

$$\theta' = \theta + \alpha \log \nabla_\theta \pi_\theta(a | s) R_t,$$

avec α le learning-step. L'algorithme a de nombreux inconvénients : il est très sensible aux paramètres initiaux et nécessite de calculer le gradient de la fonction d'évaluation, ce qui peut mener à des complications. Pour pallier ces problèmes, on peut introduire un critic network qui permet de vérifier les actions prises par la politique.

- **Actor-critic (AC) methods.** On utilise une combinaison entre la méthode de policy-gradient et celle de value-fonction qui inclut : (i) un actor qui génère chaque action et interagit avec l'environnement, et (ii) un critic qui l'évalue (selon Q ou V).

Deep Deterministic Policy Gradient (DDPG). L'actor utilise la méthode de policy gradient pour apprendre les stratégies et choisir les actions, qui sont évaluées par le critic en utilisant la fonction Q , de manière similaire au DQN. L'actor prend l'état s en entrée et génère une action a en estimant $\pi_\theta(s)$. Le critic prend l'état s et l'action choisie a en entrée, et évalue cette action en estimant $Q(s, a)$. Pour stabiliser l'apprentissage, les deux réseaux disposent de copies (target actor et target critic) qui sont mises à jour progressivement (soft update) :

$$\theta'_{target} = (1 - \tau)\theta_{target} + \tau\theta_{main}$$

où τ est très petit. Les transitions (s, a, r, s') sont stockées dans le replay buffer. Comme l'espace des actions est continu ici, la fonction Q est différentiable par rapport aux paramètres du réseau de l'actor θ^π , et on peut approximer :

$$\max_a Q(s, a) \approx \max_{\theta^\pi} Q(s, \pi_{\theta^\pi}(s)).$$

Le critic est optimisé avec une MSE entre la valeur prédite de $Q(s, a)$ et la cible, en utilisant les target networks :

$$y = r + \gamma \max_{a'} Q_{\theta^Q}(s', \pi_{\theta^\pi}(s')).$$

Et l'actor est mis à jour en maximisant la fonction Q_{θ^Q} via policy gradient, en utilisant les main networks :

$$\nabla_\theta J(\theta) = \nabla_{\theta^\pi} Q_{\theta^Q}(s, a) = \nabla_a Q_{\theta^Q}(s, a) \times \nabla_{\theta^\pi} \pi(s)$$

Cette méthode a également été appliquée à un cas multi-agents : le réseau critique reçoit des informations d'autres agents (observation globale) alors que le réseau acteur reçoit des informations que d'un seul (observation partielle).

Asynchronous Advantage Actor Critic (A3C). C'est une extension de l'algorithme AC qui utilise plusieurs threads actor-learner pour apprendre la politique et les fonctions de valeurs simultanément. Puis, ils mettent à jour le réseau global de manière périodique, ce qui accélère l'apprentissage. Comme pour D3QN, cet algorithme utilise une fonction avantage pour réduire le gradient de la variance. Contrairement à DDPG c'est une méthode on-policy, il n'y pas de replay buffer. Les paramètres du réseau sont mis à jour comme suit :

$$\theta' = \theta + \alpha \log \nabla_{\theta^\pi} \pi_\theta(a_t, s_t) A(s_t, a_t) + \beta \nabla_\theta H(\pi_\theta(s))$$

où H est une entropie et α, β sont des paramètres du réseau. Les traitements peuvent être réalisés de manière parallèle, ce qui rend l'apprentissage plus rapide. Cependant, la méthode est principalement utilisée dans des espaces d'action discrets. Ce principe a été appliqué à plusieurs situations (connaissance ou non de la carte, obstacle fixe ou dynamique) et pour plusieurs types d'entrées (images, positions, etc).

Proximal Policy Optimization (PPO). C'est une méthode de RL on-policy basée sur l'algorithme TRPO (trust region policy optimization). TRPO ajoute une contrainte sur la divergence de Kullback-Leibler entre l'ancienne et la nouvelle politique, pour éviter des changements trop brutaux. Plus particulièrement, en appliquant le théorème de policy gradient, on a :

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a | s) Q(s, a)], \quad \text{ou} \quad \nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a | s) A(s, a)],$$

qui peuvent être obtenues en différenciant :

$$J(\theta) = \mathbb{E}_{\pi_\theta} [\log \pi_\theta(a | s) A(s, a)] \quad \text{ou} \quad J(\theta) = \mathbb{E}_{\pi_\theta} \left[\frac{\pi_\theta(a | s)}{\pi_{old}(a | s)} A(s, a) \right].$$

Donc TRPO pénalise l'écart entre les politiques en utilisant :

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\frac{\pi_\theta(a | s)}{\pi_{old}(a | s)} A(s, a) - \beta D_{KL}(\pi_{old} || \pi_\theta) \right].$$

PPO simplifie cette approche en utilisant un clipping du ratio des probabilités entre les deux politiques, en le forçant à être entre $1 - \epsilon$ et $1 + \epsilon$, soit :

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\min \left(\frac{\pi_\theta(a | s)}{\pi_{old}(a | s)} A(s, a); \text{clipped} \left(\frac{\pi_\theta(a | s)}{\pi_{old}(a | s)} \right) A(s, a) \right) \right].$$

Soft Actor Critic (SAC). En plus de maximiser la récompense cumulée, cette méthode utilise une fonction objective qui permet également de maximiser l'entropie de la politique, afin de favoriser l'exploration et d'éviter les optimums locaux. En utilisant une politique stochastique et un replay buffer, elle améliore l'efficacité de l'apprentissage. Elle utilise également deux réseaux critics (double Q-learning) pour éviter la surestimation de la fonction Q .

CONCLUSION

Dans le cadre de notre étude, on s'intéresse au problème de collision avoidance entre plusieurs AGV, en supposant l'absence de communication explicite entre les agents et une prise de décision basée sur des observations locales (pas de méta-agent).

Plus précisément, l'objectif est d'apprendre aux AGV à réguler leur vitesse pour éviter les obstacles (statiques et dynamiques), tout en minimisant leur temps de mission. Ce problème s'inscrit plutôt dans un cadre continu, ce qui oriente le choix de nos modèles.

Les méthodes de type AC semblent particulièrement adaptées, car elles permettent de traiter directement des espaces d'actions continus. Au contraire, les méthodes value-based comme DQN nécessite une discrétisation de cet espace.

Par ailleurs, les méthodes off-policy sont souvent plus efficaces en terme d'apprentissage grâce à l'utilisation du replay buffer, qui stockent les transitions et permet une meilleure réutilisation des données. Les algorithmes DDPG et SAC semblent donc particulièrement pertinents. Au contraire, les méthodes on-policy comme PPO sont certes plus stables, mais moins efficaces en termes d'échantillonnage.

Finalement, cette étude suggère d'étudier les approches AC off-policy (DDPG, SAC) pour traiter notre problème. De nombreuses applications sont citées, avec différente modélisation du problème, ce qui suscite un intérêt particulier.

[3] État MIT 6.S191 : Reinforcement Learning, Amini. (2025)

SUJET

La vidéo est une retranscription d'un cours magistral d'introduction au deep learning, animé par Alexander Amini à MIT. Il présente le deep reinforcement learning (DRL), qui est une combinaison entre l'apprentissage profond et l'apprentissage par renforcement. Il s'intéresse aux deux méthodes de base : Deep Q Network (DQN) et Policy Gradient (PG).

RÉSUMÉ

L'objectif est de modéliser l'interaction entre un modèle et son environnement, dans un cadre non supervisé (ex : robotique, jeu de stratégies, etc...).

Pour rappel : un agent choisit une action, il existe et opère dans un environnement. Selon l'action choisie, l'environnement fournit une observation à l'agent : l'ensemble des observations définissent des états et des récompenses.

On définit alors 3 fonctions : R_t qui correspond au gain perçu depuis l'instant t jusqu'à la fin de l'épisode, V qui correspond au gain espéré pour l'état s_t , et Q qui correspond au gain espéré pour l'état s_t sachant que l'agent réalise l'action a_t .

$$R_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}, \quad V(s_t) = \mathbb{E}[R_t | s_t], \quad Q(s_t, a_t) = \mathbb{E}[R_t | s_t, a_t].$$

où γ est le facteur de discount et r_t est la récompense à l'instant t .

On se concentre alors sur deux méthodes principales :

- Value learning. Trouver la fonction action-valeur $Q(s, a)$ et choisir l'action qui la maximise cette fonction $a = \arg \max_a Q(s, a)$.
- Policy learning. Trouver la politique $\pi(s)$ définie sur l'ensemble des états et prédire l'action $a \sim \pi(s)$.

Deep Q networks (DQN). On peut modéliser la fonction Q avec des réseaux de neurones profonds : on prend en entrée un état s et une action a , et le modèle est entraîné pour produire la valeur $Q(s, a)$. La cible y est définie comme la combinaison de la récompense à l'état actuel avec l'espérance du retour maximal :

$$y = r + \gamma \max_{a'} Q(s', a').$$

Le modèle prédit la valeur Q pour un état s et une action a donnés. La fonction de perte utilisée $\mathcal{L}$ est donc la perte- Q qui correspond à l'espérance de la norme euclidienne de l'écart entre la cible et la prédiction.

$$\mathcal{L} = \mathbb{E} \left[\left\| \left(r + \gamma \max_{a'} Q(s', a') \right) - Q(s, a) \right\|^2 \right]$$

Notons que cette méthode sur repose un espace d'action discret uniquement, et détermine une politique déterministe (i.e. le même état engendre la même action, à chaque fois).

Policy Gradient (PG). Au lieu de retourner la valeur de $Q(s, a)$, on optimise directement la politique $\pi(s)$ en retournant la probabilité $P(a | s)$ pour chaque action. On obtient l'action choisie en échantillonnant par rapport à la loi induite : $\pi(s) \sim P(a | s)$. Cette méthode s'applique aussi bien à des espaces d'actions discrets que continus : on pourrait modéliser une distribution continue sur les résultats.

Durant l'entraînement, on réalise les étapes suivantes :

1. Initialise l'agent.
2. Exécute la politique jusqu'à l'échec.

3. Enregistre chaque triplet (s_t, a_t, R_t) .
4. Diminue la probabilité des actions qui ont donné une faible récompense (réalisées juste avant l'échec). Augmente la probabilité de celles qui ont donné une forte récompense.

La perte est définie comme le produit entre la log-vraisemblance de l'action et la récompense associée,

$$\mathcal{L} = -\log P(a_t | s_t) \times R_t.$$

Ainsi, la situation idéale serait d'avoir une récompense très grande pour une action qui a une très grande vraisemblance ($\mathcal{L} < 0$). Si on a une faible récompense pour une action qui a une très grande vraisemblance, la perte sera plus importante.

Les poids sont mis à jour par descente de gradient :

$$\theta' = \theta - \nabla \mathcal{L} = \theta + \nabla [\log P(a_t | s_t) \times R_t] .$$

Cependant, dans le cas réel, on ne veut pas vraiment simplement exécuter une politique jusqu'à l'échec, les modèles sont souvent entraînés sur des simulateurs avant d'être testés dans la vraie vie.

CONCLUSION

Dans le cadre de notre étude, on s'intéresse au problème de collision avoidance entre plusieurs AGV, en supposant l'absence de communication explicite entre les agents et une prise de décision basée sur des observations locales (pas de méta-agent).

Plus précisément, l'objectif est d'apprendre aux AGV à réguler leur vitesse pour éviter les obstacles (statiques et dynamiques), tout en minimisant leur temps de mission. Dans ce contexte, ces deux méthodes semblent être une base pertinente pour modéliser le problème

En particulier, les méthodes de PG semblent adaptées car elles permettent de traiter des espaces continus, compatibles avec le contrôle de la vitesse des agents. Au contraire, DQN nécessite une discrétisation de cet espace, ce qui constitue une approximation qui peut limiter les performances du modèle.

L'apprentissage en simulation s'intègre également dans ce cadre.

[4] Combining Deep Reinforcement Learning with Search Heuristics for Solving Multi-Agent Path Finding in Segment-based Layouts, Reijnen & al. (2020)

SUJET

Ce papier présente une méthode pour résoudre le problème de multi-agent path finding (MAPF) pour des AGV porteurs de bagage.

Leur proposition repose sur un système segmented-based, dans lequel chaque AGV est contraint de rester sur un chemin prédéfini et d'occuper des segments aux alentours, au lieu d'occuper une case de la grille. Plus particulièrement, c'est une adaptation de l'algorithme WHCA* au cas segmented-based, appelé WHCA*S.

Une variante, appelée WHCA*S-RL, permet également d'éviter des embouteillages en utilisant un modèle d'apprentissage par renforcement profond.

MODÉLISATION DU PROBLÈME

Le contexte de segmented-based est représenté comme un graphe $G = (V, E)$, où V correspond à des positions et E à des segments dirigés reliant ces positions. A chaque agent i , on associe alors une position initiale, et une position cible dans V .

La solution pour le problème MAPF détermine un plan global Π , qui est composé de chemins individuelles π_i qui permettent à chaque agent de rejoindre la position cible sans collision. L'objectif est donc de minimiser la durée moyenne de π_i pour chaque agent, afin de maximiser le nombre de bagage transporté par unité de temps.

Pour éviter les collisions, les auteurs ont introduit le concept de blocages : on empêche l'AGV d'utiliser un segment si celui-ci est déjà occupé. Pour cela, une zone limite est définie autour de l'AGV et détermine sa zone d'influence. De cette manière, on s'assure qu'il n'y a aucune collision, mais cela peut mener à des situations de deadlock, dans laquelle chaque agent est bloqué et attend l'action de l'autre.

WHCA* : Le principe consiste à trouver le chemin optimal pour une fenêtre donnée W , plutôt que pour le plan global. De cette manière, chaque agent planifie les chemins partiels de manière séquentiel, et en exécute K étapes parallèlement ($K < W$).

WHCA*S : Cependant, pour le cas segmented-based, il n'y a pas de détours possibles : si un agent est bloqué car le passage d'un autre est déjà planifié, on re-planifie, mais si un agent ne peut ni avancer, ni reculer ou rester sur place, alors on attend que la situation se débloque.

Ainsi, l'idée est d'utiliser l'algorithme de Dijkstra pour trouver le chemin le plus court, et de gérer les conflits entre les agents avec la méthode WHCA*S. Cependant, le chemin le plus court n'est pas toujours le meilleur, surtout lorsqu'il y a des risques d'embouteillages. Pour pallier ce problème, les auteurs proposent d'utiliser une méthode d'apprentissage par renforcement profond pour prendre les décisions localement.

WHCA*-RL : Dans le cadre du WHCA*-RL, on définit un processus de décision semi-markovien pour apprendre la politique. Il est défini par un tuple $\langle S, A, P, F, R \rangle$ où S est l'ensemble des états, A est l'ensemble des actions, P est la probabilité de transition, F est la distribution du temps entre deux décisions et R est la fonction de récompense.

- **Espace des états S** . Les états sont définis comme un vecteur de directions possibles $j \in \{1, \dots, J\}$ qui peuvent être prises par l'agent à sa position actuelle. Pour chaque direction j , on définit D_j le différentiel de distance avec la position cible et C_j le nombre de pas disponibles jusqu'à la destination.

- **Espace des actions A .** Les actions correspondent à l'ensemble des directions qu'un agent peut suivre à chaque position. Elles sont les mêmes quel que soit l'état, mais peuvent parfois être invalides, ce qui doit être appris par le modèle.
- **Fonction de transition P .** Elle met à jour la nouvelle position de l'agent en fonction de son état, et définit t la durée de cette action.
- **Fonction de récompense R .** Elle récompense les agents qui avancent vers la position cible et pénalisent ceux qui attendent, qui vont vers la mauvaise direction ou qui choisissent une action invalide.

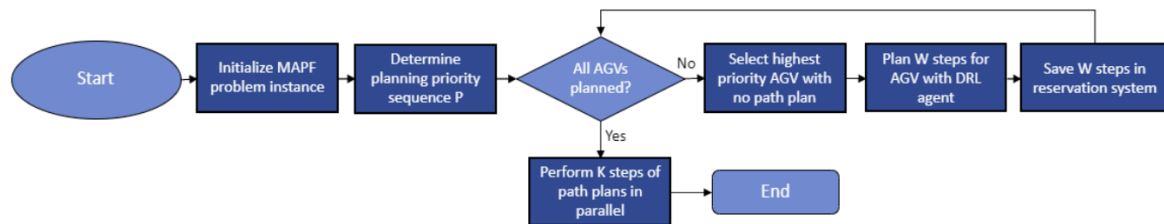


FIGURE 2 – Organigramme de la méthode WHCA*-RL. Source : [4]

Les auteurs utilisent la méthode de Proximal Policy Optimization (PPO) pour apprendre quelles sont les meilleures actions.

ENTRAÎNEMENT

L'entraînement se déroule comme suit :

1. On initialise un modèle et des scénarios d'entraînement. Ces scénarios d'entraînement sont des instances du problème sans conflits, auxquels on a assigné des positions initiales et cibles aux agents, ainsi que des chemins à priori obtenus par WHCA*S pour un nombre aléatoire d'AGV.
2. On choisit un scénario aléatoirement, et on sélectionne un AGV qui n'a pas de chemins à priori : c'est le learning AGV.
3. Le modèle prédit une action pour l'agent, en fonction de son état S : si l'action est valide, il reçoit une récompense et l'action est stockée dans le système de réservation du WHCA*S, sinon, il reçoit une pénalité et doit choisir une autre action.
4. Après la mise à jour de l'état, l'agent se déplace et son nouvel état dépend des autres plans. On continue jusqu'à avoir W actions valides, et les mises à jours des états et les récompenses associées sont utilisées pour optimiser la politique de l'algorithme PPO.

A noter que cette méthode repose sur une fonction de récompense locale, qui ne prend pas réellement en compte l'atteinte de l'objectif (ramassage ou dépôt), mais uniquement la progression vers celui-ci. Le modèle peut donc rencontrer des difficultés pour apprendre des stratégies globales optimales.

Détails : 4 500 000 steps avec 20 000 scénarios d'entraînement prédéfinis – environ 15h d'entraînement – 10 environnements parallèles – 20 steps valides nécessaires pour un AGV.

Setup : Processor Intel(R) Core(TM) i7-8850H CPU @ 2.60GHz, 32GB de RAM.

RÉSULTATS

Les méthodes WHCA*S et WHCA*S-RL ont été comparées selon le makespan, qui correspond au temps total pour que tous les AGV atteignent leur objectif, et l'average flowtime, qui correspond au temps moyen par AGV.

En moyenne, WHCA*S-RL est meilleure que WHCA*S, avec des trajets plus rapides. Plus particulièrement, la méthode est très utile lorsqu'il y a beaucoup d'AGV (>20) et que le risque de congestion est plus important. Cependant, si cette congestion est trop forte, le modèle peut beaucoup prédire l'action d'attendre et ralentir tout le système. Cela s'explique par le fait que ces situations n'ont pas été vues durant l'entraînement.

CONCLUSION

Dans le cadre de notre étude, on s'intéresse au problème de collision avoidance entre plusieurs AGV, en supposant l'absence de communication explicite entre les agents et une prise de décision basée sur des observations locales (pas de méta-agent).

Plus précisément, l'objectif est d'apprendre aux AGV à réguler leur vitesse pour éviter les obstacles (statiques et dynamiques), tout en minimisant leur temps de mission. Ce papier traite d'un problème similaire : les AGV en environnement portuaires, mais il repose sur un graphe et des choix de direction plutôt qu'un contrôle continu.

Néanmoins, l'approche proposée est intéressante pour la gestion des congestions, notamment aux intersections, en combinant planification globale avec WHCA*-S et décision locale apprise par renforcement. Toutefois, son application directe nécessite plusieurs adaptations (espace des états, espace des actions, fonction de récompense).

Finalement, cette étude suggère d'envisager une approche hybride qui combine planification globale et apprentissage local, tout en reformulant à notre cadre continu.

[5] Mobile robot path planning in dynamic environments through globally guided reinforcement learning, Wang & al. (2020)

SUJET

Ce papier traite du problème de path planning pour des robots mobiles en environnement dynamique : l'objectif est d'atteindre une destination, tout en évitant des collisions avec d'autres robots ou avec des objets dynamiques.

Les méthodes qui reposent sur de la re-planification soulèvent généralement des problèmes d'efficacité du temps d'entraînement, de pouvoir de généralisation, et de scalabilité. Comme alternative, les auteurs proposent une approche de globally guided reinforcement learning (G2RL) qui combine une global guidance et un RL-based planner.

MODÉLISATION DU PROBLÈME

L'environnement est représenté comme une grille 2D, sur laquelle est défini un ensemble d'obstacles statiques et d'obstacles dynamiques.

- La global guidance est défini comme le plus petit chemin traversable, qui permet d'atteindre la destination à partir de la position initiale. Elle n'est pas forcément unique.
- Les agents connaissent les positions des obstacles fixes et calculent la global guidance au début de chaque épisode : elle reste la même jusqu'à la fin.
- Lorsqu'il se déplace, l'agent connaît sa position sur la grille. Il connaît la position des obstacles dynamiques seulement s'ils sont dans son champ de vision.
- A chaque unité de temps, le champ de vision de l'agent est déterminé par une observation locale (cases libres, obstacles statiques, obstacles dynamiques), et par un segment local de la global guidance. Ces informations sont considérées comme des inputs à chaque unité de temps.
- A chaque unité de temps, l'agent peut soit se déplacer vers une case voisine (haut, bas, gauche, droite), ou rester à sa position (idle).

L'objectif du modèle est d'apprendre la stratégie de l'agent afin qu'il atteigne une position cible à partir d'une position initiale, en un nombre minimum d'étapes et en évitant tout collision. Pour cela, il dispose du segment local de la global guidance, de l'observation locale actuelle et de l'historique des observations locales.

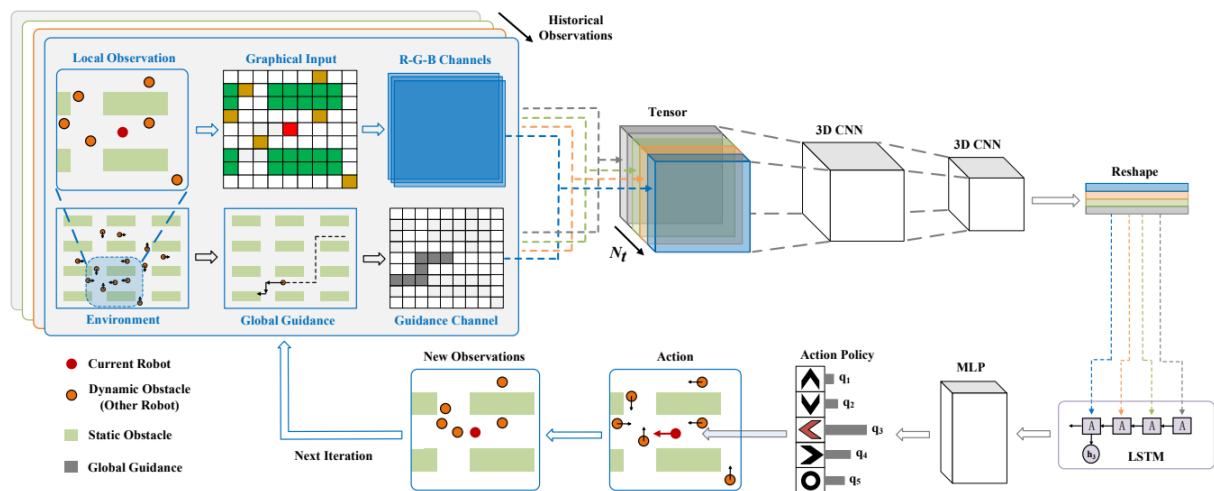


Fig. 1. The overall structure of G2RL. The input of each step is the concatenation of the transformed local observation and the global guidance information. A sequence of historical inputs is combined to build the input tensor of the deep neural network, which outputs a proper action for the robot.

FIGURE 3 – Structure de G2RL. Source : [5]

Le système G2RL, illustré en Figure 3, est composé de 4 modules principaux :

1. **Inputs.** L'observation locale est transformée en une image et ses 3 canaux (RGB) sont combinés à celui de la guidance pour composer l'input actuel. L'input final correspond à un historique des inputs.
2. **Traitement de l'espace.** On utilise des couches de CNN 3D pour extraire les caractéristiques de l'input, et on les reshape en une séquence de vecteurs 1D de la taille de l'historique.
3. **Traitement du temps.** On utilise une couche LSTM pour extraire l'information temporelle en agrégeant les vecteurs plus tard.
4. **Politique.** On utilise deux couches fully-connected pour estimer la qualité de chaque pair état-action et choisir l'action qui la maximise.

Global guidance et fonction de récompense : Le rôle principal de la global guidance est de fournir une information globale de long-terme. De cette manière, l'agent reçoit fréquemment des signaux denses, même dans de grands environnements, en construisant une fonction de récompense qui ne force pas l'agent à strictement suivre la global guidance. Elle permet de résoudre le problème de récompense sparse et améliore la généralisation à grande échelle. Plus particulièrement, l'agent reçoit une petite pénalité si il atteint une case libre qui n'appartient pas à la global guidance, une forte pénalité lorsqu'il rentre en collision, et une récompense proportionnelle au nombre de cases de la global guidance rattrapées après une déviation.

Local RL planner. L'observation partielle de l'agent est transformée en une image observation, dont 1 pixel correspond à une case de la grille : l'agent est représenté au centre de l'image, et les obstacles sont représentés de différentes couleurs, selon s'ils sont dynamiques ou statiques (input RGB). On ajoute à cela un canal qui correspond au segment de la global guidance, pour former l'input actuel. Il est ensuite transformé en vecteurs 1D auquel on applique un LSTM pour intégrer l'effet du temps, notamment la direction des obstacles. Le RL planner prend la séquence composée de ce nouvel input, ainsi que des historiques et utilise la méthode de Double Deep Q-Learning (DDQN) pour générer la meilleure action. Plus particulièrement, on obtient une valeur de la fonction Q pour chaque action, et on choisit l'action qui a la plus élevée.

Finalement, ce modèle permet à l'agent d'apprendre comment éviter des obstacles tout en suivant une global guidance. Il n'apprend pas à planifier globalement, mais à corriger localement un chemin global. Chaque agent considère uniquement une observation partielle, et n'a pas besoin de connaître les trajectoires des autres agents. De cette manière, ce modèle peut être entraîné pour un seul agent, puis être directement utilisé par plusieurs.

ENTRAÎNEMENT

Les auteurs ont testé leur approche sur 3 environnements :

- Une carte simple qui imite un entrepôt, avec des obstacles statiques et dynamiques.
- Une carte aléatoire, dans laquelle on initialise aléatoirement une certaine densité d'obstacles statiques et dynamiques.
- Une carte libre, dans laquelle on considère uniquement une certaine densité d'obstacles dynamiques.

Les cartes sont de taille 100×100 par défaut, et les obstacles dynamiques correspondent à des robots non-contrôlable, qui se déplace une case par une case.

Setup : NVIDIA GTX 1080ti GPU (Test : Intel i7-8750H CPU)

RÉSULTATS

L'approche a été comparée à des méthodes basées sur A* dynamique, avec des stratégies de replanification locale et globale.

Les résultats montrent que G2RL obtient de meilleures performances en termes de coût de déplacement et de pourcentage de détours : les trajectoires sont plus directes et nécessitent moins d'étapes pour atteindre la cible. En contrepartie, le temps de calcul est légèrement supérieur à celui des méthodes de replanification globale, tout en restant compatible avec des contraintes temps réel (fréquence d'environ 35 Hz).

Par ailleurs, l'augmentation du champ de vision et de l'historique des observations permettent d'améliorer significativement les performances.

Enfin, le modèle présente une bonne capacité de généralisation et scalabilité. En contexte multi-agents, il surpasse les méthodes décentralisées classiques telles que ORCA et PRIMAL, bien qu'avec un léger surcoût en temps de calcul.

CONCLUSION

Dans le cadre de notre étude, on s'intéresse au problème de collision avoidance entre plusieurs AGV, en supposant l'absence de communication explicite entre les agents et une prise de décision basée sur des observations locales (pas de méta-agent).

Plus précisément, l'objectif est d'apprendre aux AGV à réguler leur vitesse pour éviter les obstacles (statiques et dynamiques), tout en minimisant leur temps de mission. Ce papier présente une méthode assez proche de ce cadre : les autres agents sont modélisés comme des obstacles dynamiques, et chaque AGV prend des décisions à partir d'observations partielles.

Encore une fois, combiner une information globale, sous forme de guidance, avec une prise de décision locale apprise par renforcement, semble particulièrement pertinent. L'utilisation d'une méthode hybride permet d'améliorer les trajectoires, tout en conservant une bonne qualité de généralisation.

Dans un cadre où les AGV disposent de caméras ou autres capteurs, la représentation de l'environnement sous forme d'images locales, et leurs traitements, peuvent être des points d'inspiration dans la modélisation de notre solution.

Cependant, cette méthode repose sur un espace d'actions discret : l'agent choisit une direction, et non une vitesse. L'utilisation de l'algorithme DDQN est justifié dans leur cas, mais limiterait la précision du contrôle dans le nôtre.

Finalement, bien que cette méthode ne soit pas directement applicable, elle renforce l'intérêt d'une approche combinant guidance globale et décision locale apprise par renforcement, et pourrait être adaptée à un cadre continu.

[6] Reinforcement learning-based dynamic obstacle avoidance and integration of path planning, Choi & al. (2021)

SUJET

Ce papier présente une méthode qui combine un path planner et du reinforcement learning pour un système de navigation décentralisé, sans communication entre les agents et capable de gérer des collisions avec des obstacles dynamiques.

MODÉLISATION DU PROBLÈME

Les auteurs utilisent un differential drive robot, qui dispose d'une position (x, y) , d'une orientation ψ et d'une vitesse associée à chaque roue (v_L, v_R) . Les vitesses linéaire v et angulaire ω de l'AGV sont définies telles que :

$$v = \frac{v_L + v_R}{2}, \quad \omega = \frac{v_L - v_R}{W},$$

où W correspond à l'écart entre les deux roues de l'AGV.

Mobile robot Collision Avoidance Learning (MCAL) : Cette méthode résout le problème de collision avoidance en utilisant du deep reinforcement learning, dans un contexte décentralisé. Elle utilise un SAC pour accélérer l'apprentissage. Elle fonctionne en suivant les étapes suivantes :

- Le robot obtient les données de position et de vitesse, et celle du lidar (alentours) en interagissant avec l'environnement.
- Pour obtenir sa position globale, le robot compare les données de la carte et celles du lidar.
- La distance relative entre la position du robot et celle de la cible correspond à l'input fourni au RL agent.
- Le RL agent reçoit des informations concernant la distance relative, les données du lidar et la vitesse du robot. Il retourne les vitesses linéaire et angulaire.
- Le robot se déplace en contrôlant sa vitesse linéaire et angulaire, obtenu à partir du RL agent

Les auteurs formulent le problème de collision avoidance dans le cadre de l'apprentissage par renforcement.

- **Espace des états S .** L'état correspond aux données du lidar, qui combinent les distances avec l'environnement aux alentours, les vitesses linéaire v et angulaire ω , et les distances relatives de x et y de la position du robot à la position cible,

$$s_t = \left[s_t^{lidar}, s_t^{goal}, s_t^{speed} \right],$$

avec $s_t^{lidar} = [o_l^{t-2}, o_l^{t-1}, o_l^t]$ correspond aux distances avec les obstacles sur les trois derniers temps, $s_t^{goal} = o_g^t = [x, y]$ correspond à la position relative du robot par rapport à l'objectif et $s_t^{speed} = o_s^t = [v, \omega]$ correspond aux vitesses du robot.

- **Espace des actions A .** Le comportement du robot est modélisé comme continu pour permettre un déplacement fluide et différente manière d'éviter les collisions,

$$a_t = [v, \omega],$$

avec $v \in [0, v_{max}]$ et $\omega \in [-\omega_{max}, \omega_{max}]$.

- **Fonction de récompense R .** L'objectif du robot est d'atteindre la position cible (P_x, P_y) . Ici, l'orientation n'est pas pris en compte car le differential drive robot peut facilement pivoter. La fonction de récompense correspond à la somme d'une fonction R_g qui valorise le fait de se rapprocher de la cible, une fonction R_c qui pénalise les collisions et R_ω qui pénalise les rotations trop importantes (cela est lié aux limites du robot, dont le contrôle est plus difficile lorsqu'il pivote rapidement),

$$R = R_g + R_c + R_\omega.$$

- **Condition de fin.** Un épisode prend fin dans 3 cas : le robot a atteint la position cible, le robot est entré en collision avec un obstacle ou l'épisode a exécuté plus de 2000 étapes.

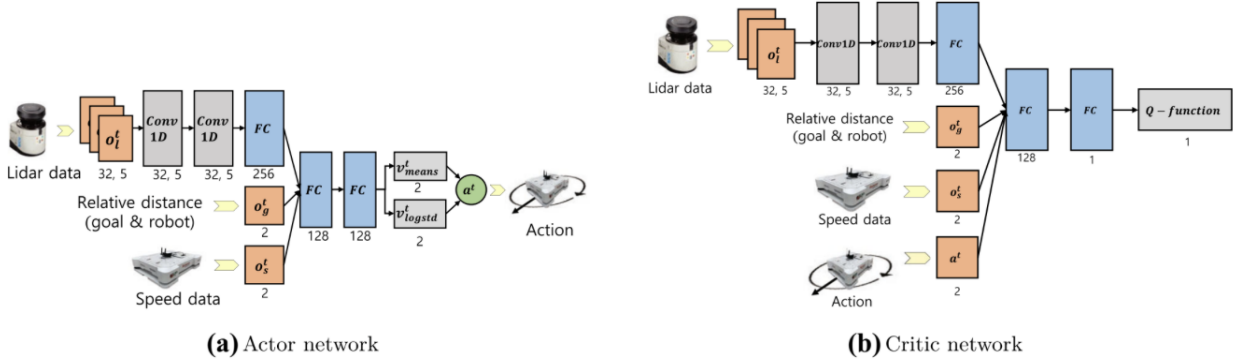


FIGURE 4 – Actor et critic networks pour le MCAL. Source : [6]

SAC : Pour appliquer l'algorithme SAC pour le problème de collision avoidance, on définit un actor network qui construit la politique et un critic network qui construit la fonction Q pour l'évaluer.

- **Actor network.** Le réseau consiste en 2 couches de convolution, 3 couches fully-connected, 2 fonctions d'activation non-linéaires ReLU et 1 fonction d'activation linéaire. Les données du lidar (512x3) sont passées dans 2 couches convolutionnelles pour en extraire les évolutions de l'environnement (mouvement des obstacles). On ajoute ensuite à cela la position relation (x, y) du robot ainsi que les vitesses v et ω , et on les passe dans 2 couches fully-connected. Le réseau retourne une vitesse moyenne v_μ^t et le logarithme de l'écart-type $v_{\log \sigma}^t$. On a alors :

$$a_t \sim \mathcal{N}(v_\mu^t, v_{\log \sigma}^t).$$

- **Critic network.** L'architecture est similaire, sauf qu'on rajoute l'action obtenue par le actor network a_t et on retourne la Q-value.

Les auteurs ont tenté plusieurs algorithmes et on choisit SAC, car il peut réutiliser l'ensemble des informations collectés durant l'entraînement pour l'apprentissage du réseau de neurones profond : c'est une méthode off-policy. SAC optimise la fonction objective :

$$J(\theta) = \sum_{t=0}^T \mathbb{E}_{\pi_\theta} [r(s_t, a_t) + \alpha H(\pi_\theta(\cdot | s_t))].$$

L'objectif est de maximiser l'espérance de la récompense cumulée, tout en encourageant l'exploration. De cette manière, les agents agissent plus aléatoirement, étant donné que la fonction gaussienne est plutôt plate.

SAC a été comparé à l'algorithme PPO, avec pour conclusion que SAC est plus rapide. Cela s'explique par le fait que SAC garde les informations dans un replay buffer (s, a, r, s') .

ENTRAÎNEMENT

Les auteurs ont entraîné le modèle sur deux simulateurs : (i) un Stage simulator qui permet d'entraîner vite, mais qui ne prend pas en compte des facteurs cinétiques tels que l'inertie ou les frictions, et (ii) le Gazebo Simulator qui est plus lent, mais qui prend en compte ces facteurs, et est donc plus proche de la réalité. Ainsi, le modèle est d'abord entraîné sur la simulation légère, puis affiné avec la simulation plus complexe.

- Le Stage simulator est configuré dans un espace circulaire de 30m de diamètre, avec 7 obstacles statiques et 4 obstacles dynamiques, qui se déplacent de manière aléatoire entre 0.5m/s et 1m/s

sans stratégie d'évitement particulière. 4 robots sont également déployés simultanément, et correspondent à des obstacles dynamiques les uns pour les autres, ce qui permet d'accélérer l'apprentissage.

- Le Gazebo simulator est configuré dans un espace carré de 20×20m, avec des obstacles statistiques et dynamiques placés de manière aléatoire, ainsi que 4 robots qui apprennent leur politique simultanément.

Détails : 20 000 épisodes sur le Stage simulator - 3 000 épisodes sur Gazebo.

RÉSULTATS

Les robots ont plutôt bien appris à se déplacer sans collision, en évitant les obstacles statiques et dynamiques, avec une diminution de la vitesse linéaire et une augmentation de la vitesse angulaire significatives.

Cependant, les auteurs ont remarqué que le robot rencontre des difficultés majeures lorsque les positions initiale et cible sont séparées par un mur : cela est dû à deux objectifs conflictuels, dont l'un consiste à minimiser la distance avec l'objectif et l'autre à maintenir une distance de sécurité avec l'obstacle. En effet, le RL ici permet de prédire des comportements locaux basés sur la perception du robot, mais n'assure pas le chemin optimal. Pour résoudre ce problème, les auteurs proposent d'ajouter un path planner.

Mobile robot Collision Avoidance Learning with Path (MCAL_P) : Pour améliorer la méthode MCAL, les auteurs intègrent une méthode classique de path planning basé sur un look-ahead point : c'est un point mobile maintenu à une certaine distance du robot, et qui fait office de point cible en entrée du MCAL.

- Le robot obtient les données de position et de vitesse, et celle du lidar (alentours) en interagissant avec l'environnement.
- Pour obtenir sa position globale, le robot compare les données de la carte et celles du lidar.
- Une méthode classique de path planning (comme l'algorithme A*) donne le chemin optimal entre la position du robot et le point cible, à partir des informations de la carte, du point cible et de la position du robot.
- Un look-ahead point est trouvé, à partir des inputs du chemin optimal et de la position du robot sur la carte.
- La distance relative entre la position du robot et celle du point cible sur le chemin correspond à l'input fourni au RL agent.
- Le RL agent reçoit des informations concernant la distance relative, les données du lidar et la vitesse du robot.
- Le robot se déplace en contrôlant sa vitesse linéaire et angulaire, obtenu à partir du RL agent

Le RL agent correspond exactement au RL présenté et entraîné précédemment.

Le modèle a été entraîné avec le même environnement sur les mêmes scénarios, afin de comparer MCAL et MCAL_P. MCAL_P résout le problème mentionné plus tôt. Il fonctionne alors mieux, car il combine un chemin optimal global avec une prise de décision locale.

CONCLUSION

Dans le cadre de notre étude, on s'intéresse au problème de collision avoidance entre plusieurs AGV, en supposant l'absence de communication explicite entre les agents et une prise de décision basée sur des observations locales (pas de méta-agent).

Plus précisément, l'objectif est d'apprendre aux AGV à réguler leur vitesse pour éviter les obstacles (statiques et dynamiques), tout en minimisant leur temps de mission. Ce papier correspond à notre cadre : les autres agents sont modélisés comme des obstacles dynamiques, et chaque AGV régule leur vitesse à partir d'observations partielles, pour éviter les collisions.

Cette méthode pourrait être appliquée à notre problème avec certaines adaptations, notamment concernant la modélisation de la dynamique des AGV. Encore une fois, combiner une planification globale et une prise de décision locale semble très pertinent.

De plus, l'utilisation de SAC, qui est une méthode off-policy, améliore l'efficacité de l'apprentissage. Ayant été validée à la fois en simulation et en environnement réel, cette approche est particulièrement crédible et peut constituer une source d'information pour notre étude.

Finalement, cette méthode semble prometteuse, mais nécessite d'être évaluée dans notre cadre spécifique, notamment en termes de scalabilité et de robustesse en environnement multi-agents denses.

[7] Autonomous Navigation and Collision Avoidance for AGV in Dynamic Environments : An Enhanced Deep Reinforcement Learning Approach With Composite Rewards and Dynamic Update Mechanisms, Huang & al. (2024)

SUJET

Ce papier propose une méthode de deep reinforcement learning (DRL) pour permettre à des AGV de naviguer en évitant les obstacles dans un environnement complexe et dynamique. Elle repose sur une version améliorée de TD3 (twin delayed DDPG), afin de stabiliser l'apprentissage et d'améliorer l'efficacité et la robustesse.

Les auteurs apportent principalement 3 contributions :

1. Ils modélisent le problème de navigation en temps-réel et d'évitement d'obstacle comme un processus de décision de Markov (MDP).
2. Ils proposent une fonction de récompense composite pour résoudre le problème de reward sparsity.
3. Ils proposent un TD3 amélioré avec une atténuation adaptative du bruit et une mise à jour différée dynamique

MODÉLISATION DU PROBLÈME

L'AGV est modélisé comme un disque centré en $(x(t), y(t))$ à chaque unité de temps t . On note également $\theta(t)$ l'orientation de l'AGV, $\phi(t)$ l'angle de ses roues, $v(t)$ sa vitesse linéaire et $\omega(t)$ sa vitesse angulaire. Durant sa navigation, il est soumis à plusieurs contraintes : limites de l'environnement, présence d'obstacles (statiques ou dynamiques) et contraintes physiques.

L'objectif de l'AGV est de déterminer une trajectoire optimale pour aller d'une position de départ à une position cible (x_g, y_g) , sans entrer en collision.

Un processus de décision de Markov (MDP) modélise la prise de décision en environnement incertain. Il est représenté par le tuple (S, A, P, R, γ) où S est l'ensemble des états, A est l'ensemble des actions, P est la probabilité de transition entre les états, R est la fonction de récompense et γ est le facteur de discount. La probabilité qu'un agent passe d'un état s_1 à un état s_t peut s'écrire comme :

$$p_{\theta}(\tau) = p(s_1) \prod_{t=1}^T p_{\pi_{\theta}}(a_t | s_t) p(s_{t+1} | s_t, a_t),$$

où $\tau = \{s_1, a_1, \dots, a_{T-1}, s_T\}$ est une trajectoire spécifique d'un épisode, en T pas.

L'objectif est donc de maximiser la récompense cumulée actualisée, qui est une somme des récompenses pondérées par γ . Si γ est élevé, l'agent privilégie le long-terme.

- **Espace des états S .** Soit S_n l'état de l'AGV, S_{laser} le résultat du scan d'un radar laser 2D pour détecter les obstacles et S_0 la distance et l'angle du plus proche obstacle. L'espace des états S correspond à un vecteur 6D.

$$S_n = [x_g - x, y_g - y, \theta, \phi], \quad S_{laser} = [d_0, \theta_0, \dots, d_{350}, \theta_{350}], \quad S = [S_n, S_0].$$

- **Espace des actions A .** L'AGV est flexible et est capable de se déplacer dans n'importe quelle direction, de manière continue, en ajustant sa vitesse et son orientation pour atteindre son objectif,

$$A = [v, \omega]$$

où est v la vitesse linéaire actuelle et ω est la vitesse angulaire actuelle.

- **Fonction de récompense R .** La fonction de récompense est décomposée en une récompense de distance R_D qui valorise le fait de se rapprocher de la position cible, une récompense d'action

R_A qui encourage à l'atteindre rapidement, une récompense cible R_G qui s'active lorsque l'AGV l'a atteint et une pénalité d'obstacle P_O qui entraîne l'AGV à maintenir une distance de sécurité. On a alors :

$$R = \beta_1 R_D + \beta_2 R_A + \beta_3 R_G + \beta_4 P_O.$$

Avec cette fonction de récompense, l'objectif est d'atteindre la cible, rapidement et sans collision. Les poids ajustables permettent de prioriser ou non certains aspects : par exemple, si on sait qu'il y a peu d'obstacles, on peut augmenter β_1 et β_2 pour privilégier le fait d'aller vite.

- **Condition de fin** : Un épisode prend fin dans 3 cas : le robot a atteint la position cible, le robot est entré en collision avec un obstacle, ou le nombre de steps maximum a été atteint.

TD3 : DDPG est une méthode de deep reinforcement learning basé sur un algorithme actor-critic. L'actor prend l'état s et génère une action continue a en estimant $\pi(s)$. Le critic prend l'état s et l'action choisie a , et évalue cette action en estimant $Q(s, a)$.

Pour éviter la surestimation de la fonction Q , TD3 introduit un deuxième critic : on a deux réseaux Q qui estiment $Q_1(s, a)$ et $Q_2(s, a)$, et on prend $\min(Q_1(s, a); Q_2(s, a))$.

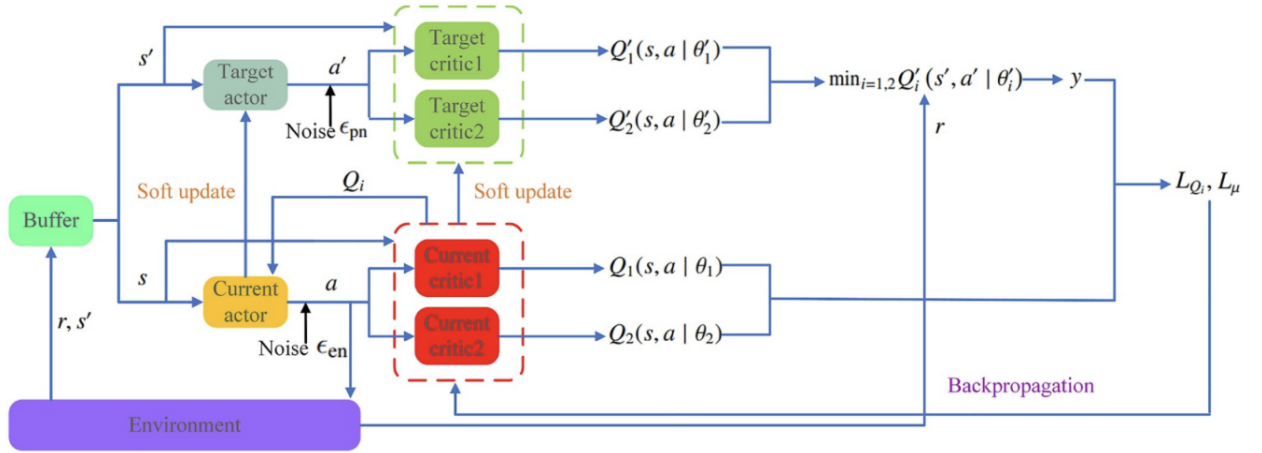


FIGURE 5 – Procédure du TD3 amélioré. Source : [7]

Amélioration de TD3 : Pour améliorer la vitesse d'apprentissage et la robustesse des résultats, les auteurs introduisent deux éléments.

- Atténuation adaptative du produit : Pour forcer l'exploration et éviter les optimums locaux, TD3 ajoute un bruit ϵ aux actions de l'actor : $a = \pi(s) + \epsilon$. Pour avoir un équilibre entre exploration et exploitation, les auteurs proposent d'utiliser un bruit gaussien, dont la variance diminue avec le temps t :

$$\epsilon \sim N(0, \sigma^2(t)).$$

De cette manière, le modèle explore beaucoup au début, puis exploite ce qu'il a appris à la fin.

- Mécanisme de mise à jour différé : TD3 met à jour les paramètres du critic à chaque étape, mais le fait de manière périodique pour l'actor. En effet, le critic est très instable au début, on souhaite donc éviter que l'actor apprenne d'un critic encore mauvais. Les auteurs proposent d'ajuster la fréquence de mise à jour des paramètres de l'actor avec la méthode exponentially weighted moving average (EWMA) : elle absorbe rapidement le bruit, ce qui permet de lisser les valeurs de la fonction de perte. Soit y la cible du réseau Q , on a L_Q la perte et S le lissage :

$$L_Q = (Q_\theta(s, a) - y)^2, \quad S(t) = \alpha L(t) + (1 - \alpha) S(t - 1),$$

où α est le facteur de lissage. Il calcule ensuite un changement moyen sur h étapes :

$$\Delta S(t) = \frac{S(t) - S(t - 1)}{S(t - 1)}, \quad \Delta \bar{S}(i, i + h) = \frac{1}{h} \sum_{t=1}^{i+h} \Delta S(t)$$

où i est un entier multiple de h . Finalement, la fréquence de changement est définie comme telle :

$$f = \left\lfloor \frac{\Delta \bar{S}}{10} \right\rfloor + 1.$$

Si $\Delta \bar{S}$ est petit, f aussi : la fonction de perte est assez stable donc on met à jour fréquemment.

Finalement, pour assurer la stabilité de l'apprentissage, étant donné qu'on dispose de mesures de types différents (distance, angle), l'espace des états est normalisé.

ENTRAÎNEMENT

A chaque épisode, on initialise aléatoire les obstacles (statiques et dynamiques) et leur taille, la position de l'AGV et la position cible, sur un espace d'aire 10×10 . L'objectif est d'atteindre un fort pouvoir de généralisation.

Détails : 2 000 épisodes d'entraînement - 1 000 épisodes de test.

Setup : Windows 10 X64 Professional machine avec un 2.5-GHz Intel Core i7 CPU et 16 GB de RAM.

RÉSULTATS

La méthode a un plus haut taux de succès, moins de collisions et moins de timeouts que les algorithmes DDPG, SAC et TD3. Quand la vitesse augmente, les performances se dégradent mais celle-ci reste tout de même la meilleure. La convergence est également plus stable, avec une loss plus lisse et une récompense plus élevée à la fin. Cependant, l'apprentissage est un peu plus lent au début.

CONCLUSION

Dans le cadre de notre étude, on s'intéresse au problème de collision avoidance entre plusieurs AGV, en supposant l'absence de communication explicite entre les agents et une prise de décision basée sur des observations locales (pas de méta-agent).

Plus précisément, l'objectif est d'apprendre aux AGV à réguler leur vitesse pour éviter les obstacles (statiques et dynamiques), tout en minimisant leur temps de mission. Ce papier correspond à notre cadre : les autres agents sont modélisés comme des obstacles dynamiques, et chaque AGV régule leur vitesse à partir d'observations partielles, pour éviter les collisions.

Cette méthode pourrait être appliquée à notre problème avec peu d'adaptation. L'utilisation du TD3 amélioré apparaît très pertinent pour traiter les environnements dynamiques et continus.

Néanmoins, cette méthode soulève des questions en terme de coûts d'apprentissage. Bien que les performances obtenues soient meilleures que pour les méthodes de référence, les auteurs ne précisent pas les temps d'exécution, ce qui constitue un critère important dans notre projet.

Finalement, cette méthode semble prometteuse, mais nécessite une validation de sa faisabilité et de son efficacité dans notre contexte.

[8] Realtime Collision Avoidance for Mobile Robots in Dense Crowds using Implicit Multi-sensor Fusion and Deep Reinforcement Learning, Liang & al. (2020)

SUJET

Ce papier traite du problème de navigation dans un environnement dense et dynamique, en particulier au sein de foules de piétons. Les auteurs proposent une approche de deep reinforcement learning (DRL) qui utilise l'algorithme de proximal policy optimization (PPO), afin d'apprendre une politique de navigation sans collision.

L'approche est end-to-end : elle apprend directement une fonction qui relie les observations issues de capteurs aux commandes de vitesses du robot, sans étape intermédiaire.

Les principales contributions sont les suivantes :

- Une méthode d'apprentissage end-to-end qui repose sur une fusion de capteurs, permettant d'exploiter conjointement les informations pour améliorer la perception de l'environnement.
- L'apprentissage d'une politique de navigation capable de générer des trajectoires fluides et sans collision, robuste aux bruits de perception et aux occlusions, même en forte densité de piétons.
- L'utilisation de simulations 3D réalistes dans un cadre sim-to-real, permettant un transfert efficace de la politique apprise vers des robots réels.

MODÉLISATION DU PROBLÈME

Le robot est soumis à des contraintes non-holonomes, et ne dispose d'aucune connaissance globale sur son environnement. Il perçoit uniquement un environnement local par le biais de ces capteurs. Plus particulièrement, à chaque instant t , le robot reçoit un vecteur d'observations $\mathbf{o}^t$, à partir duquel il calcule une commande de vitesse. Son objectif est d'atteindre une position cible, en évitant toute collision avec des obstacles statiques ou dynamiques. La responsabilité de l'évitement des piétons repose uniquement sur le robot.

La méthode nécessite l'utilisation de plusieurs capteurs simultanément. Un lidar fournit les informations de distance aux obstacles, mais est limité pour distinguer la nature des objets ou leurs dynamiques. Pour pallier ces limitations, les données sont combinées avec celles d'une caméra (RGB ou profondeur), capable de capturer des informations sur la forme, le mouvement et les interactions des obstacles statiques et dynamiques, ainsi que les piétons.

Le problème est formulé comme un Partially Observable Markov Decision Process (POMDP), représenté par le tuple $\langle S, A, P, R, \Omega, O \rangle$, où S est l'ensemble des états, A est l'ensemble des actions, P est la probabilité de transition entre les états, R est la fonction de récompense, Ω est l'espace des observations et O est la distribution de l'observation selon l'état du système.

- **Espace des observations Ω** : L'observation est décomposée en 4 éléments :

$$\mathbf{o}^t = [\mathbf{o}_{lid}^t, \mathbf{o}_{cam}^t, \mathbf{o}_g^t, \mathbf{o}_v^t],$$

où $\mathbf{o}_{lid}^t$ les données brutes du lidar, $\mathbf{o}_{cam}^t$ les données brutes de la caméra, $\mathbf{o}_g^t$ la position relative à l'objectif du robot, et $\mathbf{o}_v^t$ les vitesses actuelles du robot. Plus particulièrement :

$$\mathbf{o}_{lid}^t = \{l \in \mathbb{R}^{512} : 0 < l_i < 4\} \quad \text{et} \quad \mathbf{o}_{cam}^t = \{C \in \mathbb{R}^{150 \times 120} : 1.4 < C_{ij} < 5\}.$$

- **Espace des actions A** . L'espace des actions correspond aux commandes de vitesse du robot, composée d'une vitesse linéaire v^t et d'une vitesse angulaire ω^t :

$$\mathbf{a}^t = [v^t, \omega^t].$$

- **Fonction de récompense R .** La fonction de récompense encourage le robot à atteindre sa cible tout en évitant les collisions et en produisant des trajectoires fluides. Elle est définie comme la somme de plusieurs termes :

$$r^t = r_g^t + r_c^t + r_{osc}^t + r_{safedist}^t.$$

où r_g^t récompense la progression vers l'objectif, r_c^t pénalise fortement les collisions avec les obstacles, r_{osc}^t pénalise les comportements oscillatoires et $r_{safedist}^t$ pénalise les situations où le robot est trop proche d'un obstacle.

Les récompenses associées à l'atteinte de l'objectif et à l'évitement de collision sont priorisées, tandis que les termes liés à la fluidité de la navigation et à la distance de sécurité jouent un rôle de régularisation.

A chaque instant, la nouvelle action à partir de la politique entraînée : $\mathbf{a}^t \sim \pi_\theta(\mathbf{a}^t | \mathbf{o}^t)$. L'objectif est d'apprendre la politique qui minimise le temps moyen pour arriver à la position cible :

$$\arg \min_{\pi_\theta} \mathbb{E} \left[\frac{1}{N} \sum_{i=1}^N t_i^g \mid \pi_\theta \right],$$

où t_i^g est le temps d'arrivée à l'objectif pour l'épisode i .

PPO : Le problème d'optimisation est résolu à l'aide de l'algorithme PPO, une méthode de policy gradient, adaptée aux espaces d'action continus. PPO améliore la stabilité de l'apprentissage en contraignant les mises à jour de la politique avec la divergence de Kullback-Liebert (KL). Plus particulièrement, la perte surrogée L^{PPO} est optimisée :

$$L^{PPO}(\theta) = \sum_{t=1}^{T_{max}} \frac{\pi_\theta(\mathbf{a}^t | \mathbf{o}^t)}{\pi_{old}(\mathbf{a}^t | \mathbf{o}^t)} \hat{A}^t - \beta D_{KL}(\pi_{old} \parallel \pi_\theta) + \xi \max(0; D_{KL}(\pi_{old} \parallel \pi_\theta) - 2KL_{target})^2$$

où $\hat{A}^t$ est un estimateur de la fonction avantage, et KL_{target} , β et ξ sont des hyper-paramètres du modèle.

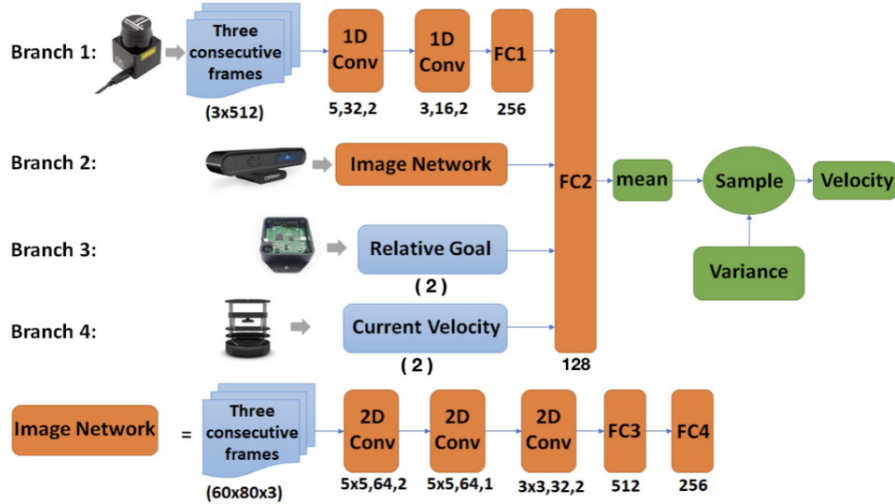


FIGURE 6 – Architecture du réseau de CrowdSteer, composé de 4 branches. Source : [8]

Le réseau est composé de 4 branches, chacune dédiée à un composant de l'observation $\mathbf{o}^t$. Cette architecture est motivée par l'hétérogénéité des données en entrées (lidar 1D et images 2D), qui ne peuvent pas être traitées simultanément dès les premières couches.

- Les branches 1 et 2 traitent les données du lidar et de la caméra. Chaque branche utilise des couches de convolutions (Conv1D ou Conv2D) pour extraire les caractéristiques spatiales, à partir de plusieurs frames consécutives. Cela permet notamment de capturer les dynamiques des obstacles.
- Les sorties de ces branches sont projetées dans un espace de dimension commune avec des couches FC, puis concaténées avec les autres composantes de l’observation : la distance relative et la vitesse actuelle.
- L’ensemble est traité par une couche FC qui produit les paramètres de la distribution de l’action. Plus particulièrement, le réseau prédit la vitesse moyenne, ainsi que le logarithme de l’écart-type $\mathbf{a}_{\log std}^t$.

Afin de respecter les contraintes cinématiques du robot, une fonction sigmoïde est appliquée à la vitesse linéaire et une fonction tanh à la vitesse angulaire. Finalement, l’action est échantillonnée selon une distribution gaussienne :

$$\mathbf{a}^t \sim \mathcal{N}(\mathbf{a}_{mean}^t, \mathbf{a}_{\log std}^t),$$

où les paramètres de la distribution sont appris de manière end-to-end.

ENTRAÎNEMENT

La politique est entraînée progressivement, en complexifiant les scénarios petit à petit, afin de faciliter l’apprentissage et améliorer la généralisation (curriculum learning) :

- **Scénario statique** : 2 robots partagent la même politique et doivent atteindre des objectifs fixes en évitant des obstacles simples. Les objectifs et les positions initiales sont ensuite attribués aléatoirement.
- **Scénario avec obstacles statiques et dynamiques** : Le robot doit atteindre son objectif en navigant à travers des piétons statiques et dynamiques. Le mouvement des piétons est généré via des points de passage, pour simuler un comportement réaliste.
- **Scénario avec occlusions** : Le robot doit atteindre son objectif en navigant dans un environnement de type couloirs, avec des visages serrés et des piétons partiellement visibles. Cela permet d’apprendre à réagir rapidement face à des obstacles qui apparaissent tardivement.

Afin de favoriser le transfert vers le réel, les auteurs utilisent des environnements 3D réalistes. Pour éviter du sur-apprentissage à des scénarios spécifiques, des scénarios simples sont maintenus en parallèle durant l’entraînement.

Détails : 6 jours d’entraînement (dû à la dimension de la caméra profondeur 3D) – 200 itérations/scénario.

Setup : Intel Xeon 3.6GHz CPU et Nvidia GeForce RTX 2080Ti GPU.

RÉSULTATS

La politique entraînée est évaluée sur plusieurs scénarios plus complexes que ceux utilisés durant l’entraînement. Ces scénarios incluent notamment des environnements étroits, denses et avec occlusions. Les auteurs introduisent la notion de Least passage Space (LPS), qui correspond à l’espace minimal disponible pour naviguer entre les obstacles.

Les performances de CrowdSteer sont comparées à plusieurs approches : (i) DWA, une méthode classique basée sur un lidar et un planificateur global, (ii) une méthode basée sur une caméra de profondeur entraînée avec PPO, et (iii) une méthode de l’état de l’art [9] basé sur un lidar 2D, qui suppose une coopération des agents pour éviter les collisions. L’évaluation repose alors sur plusieurs métriques : le taux de succès, la longueur moyenne des trajectoires, le temps moyen pour atteindre l’objectif, et la vitesse moyenne.

CrowdSteer obtient des performances similaires à DWA, tout en ne nécessitant ni carte globale ni planificateur. Cependant, son avantage devient significatif lorsque le nombre d’agents dynamiques augmente, où DWA voit ses performances chuter. Comparé à la méthode (iii), CrowdSteer présente des résultats

similaires en termes de longueur de trajectoire, de temps moyen et de vitesse moyenne, mais se distingue par un taux de succès supérieur. En effet, la méthode (iii) est conçue pour l'évitement de collision en supposant une coopération entre les agents, ce qui peut conduire à des blocages ou à des comportements oscillatoires. Par ailleurs, CrowdSteer génère des trajectoires plus stables et plus fluides.

Enfin, les expérimentations sur robots réels confirment la capacité du modèle. Toutefois, certaines limites apparaissent dans des environnements très denses, très contraints, ou en présence de surface réfléchissantes, où les performances peuvent se dégrader.

CONCLUSION

Dans le cadre de notre étude, on s'intéresse au problème de collision avoidance entre plusieurs AGV, en supposant l'absence de communication explicite entre les agents et une prise de décision basée sur des observations locales (pas de méta-agent).

Plus précisément, l'objectif est d'apprendre aux AGV à réguler leur vitesse pour éviter les obstacles (statiques et dynamiques), tout en minimisant leur temps de mission.

L'approche proposée présente des résultats solides, avec une bonne robustesse et capacité de généralisation. L'utilisation de la fusion multi-capteurs et de l'apprentissage end-to-end constitue un apport intéressant pour traiter des entrées hétérogènes.

Toutefois, l'approche est essentiellement centrée sur un agent unique devant des obstacles dynamiques, et la modélisation comme un système multi-agent coopératif semble plus adaptée. Par ailleurs, la complexité du modèle et le temps d'entraînement élevé peuvent constituer un frein.

Ainsi, bien que cette approche apporte des idées pertinentes, elle semble surdimensionnée pour notre étude. Des méthodes plus simples ou plus orientées multi-agents pourraient être mieux adaptées.

[10] Reinforcement Learned Distributed Multi-Robot Navigation With Reciprocal Velocity Obstacle Shaped Rewards, Han & al. (2022)

SUJET

Ce papier traite du problème de navigation sans collision, dans un système multi-agent décentralisé, où chaque robot prend ses décisions à partir d'information locales. Il propose de combiner les approches de type Velocity Obstacles, qui fournissent une représentation explicite des contraintes de collision, avec du deep reinforcement learning (DRL), afin d'apprendre une politique de navigation efficace.

Les auteurs apportent principalement 3 contributions :

- Ils donnent une représentation unifiée de l'environnement, dans laquelle les obstacles statiques sont modélisés par des vecteurs VO et les agents dynamiques sont modélisés par des vecteurs Reciprocal Velocity Obstacles (RVO), permettant de capturer explicitement les interactions de collision.
- Ils utilisent un réseau de neurones récurrent bidirectionnel de type Gated Recurrent Units (BiGRU), capable de produire des actions directement à partir des caractéristiques de l'environnement.
- Ils construisent une fonction de récompense basée sur les zones de collision (VO/RVO) ainsi que sur le temps estimé avant collision, afin d'encourager des comportements d'évitement réciproque.

MODÉLISATION DU PROBLÈME

Les approches basées sur les VO et leurs extensions (RVO, ORCA, NH-ORCA, PRVO) permettent de modéliser explicitement les contraintes de collision entre agents. En parallèle, les méthodes de DRL apprennent directement des politiques de navigation à partir d'expériences, mais peinent souvent à gérer des entrées de taille variable et des interactions complexes.

Les auteurs considèrent un système composé de robots mobiles qui se déplacent dans un environnement partagé, selon des directions x et y . Le système est décentralisé : les robots ne communiquent pas entre eux mais perçoivent leur environnement local.

- Chaque robot connaît son rayon, sa vitesse actuelle, son orientation et sa vitesse désirée.
- Les robots voisins sont décrits par leurs rayons, leur positions relatives et leurs vitesses. Ils sont représentés comme des vecteurs RVO.
- Les obstacles statiques sont représentés comme des VO.

L'ensemble de ces informations est utilisé en entrée du réseau de neurones qui apprend la politique, apprise à l'aide de l'algorithme proximal policy optimization (PPO).

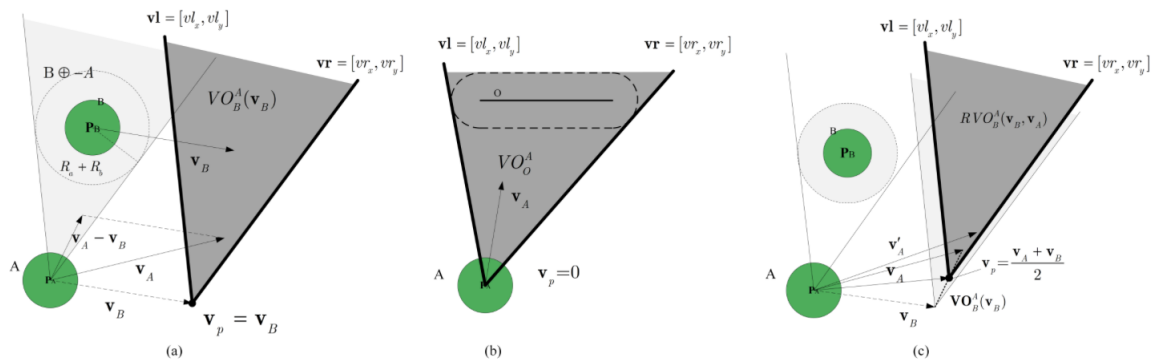


FIGURE 7 – VO et RVO pour un agent dynamique et un obstacle statique linéaire. Source : [10]

RVO : Considérons deux robots circulaires A et B , de rayon R_A et R_B , dont les positions et vitesses sont respectivement $\mathbf{p}_A$, $\mathbf{p}_B$, $\mathbf{v}_A$ et $\mathbf{v}_B$. Le VO de A induit par B correspond à l'ensemble des vitesses de A qui

entraînaient une collision avec B , et est donné par :

$$VO_B^A(\mathbf{v}_B) = \{\mathbf{v}_A \mid \lambda(\mathbf{p}_A, \mathbf{v}_A - \mathbf{v}_B) \cap B \oplus -A \neq \emptyset\}$$

où $\lambda(\mathbf{p}, \mathbf{v})$ correspond à la demi-droite issue de $\mathbf{p}$ dans la direction $\mathbf{v}$, et $\oplus$ est la somme de Minkowski. Plus particulièrement, il s'agit des vitesses, telles que la trajectoire relative de A par rapport à B intersecte le volume occupé par B (élargi par le rayon de A). Choisir une vitesse en dehors de cette région garantit l'absence de collision.

Dans le cas des obstacles statiques, on utilise une construction similaire. Cependant, s'ils bloquent totalement l'accès à l'objectif, une stratégie de navigation globale est nécessaire.

Dans le cas multi-agents, les RVO introduisent un principe d'évitement réciproque, selon lequel chaque robot ajuste sa vitesse de manière symétrique. De cette manière, chaque robot contribue de manière équitable à l'évitement de collision.

$$RVO_B^A(\mathbf{v}_B, \mathbf{v}_A) = \{\mathbf{v}'_A \mid 2\mathbf{v}'_A - \mathbf{v}_A \in VO_B^A(\mathbf{v}_B)\}$$

Cela correspond à translater le cône de collision dans l'espace des vitesses : le sommet n'est plus $\mathbf{v}_B$ mais $\frac{\mathbf{v}_A + \mathbf{v}_B}{2}$.

Ainsi, les VO et RVO peuvent être représentés par un vecteur $\mathbf{c} = [\mathbf{v}_p, \mathbf{vl}, \mathbf{vr}]$ où $\mathbf{v}_p = [v_x, v_y]$ correspond au sommet du cône, et $\mathbf{vl} = [vl_x, vl_y]$ et $\mathbf{vr} = [vr_x, vr_y]$ correspondent aux demi-droites gauche et droite.

Le problème consiste à faire naviguer plusieurs robots sans collision en choisissant, à chaque instant t , une vitesse optimale. Pour un robot i de rayon R_i , de position $\mathbf{p}_t^i$ et de vitesse $\mathbf{v}_t^i$, l'objectif est de rester au plus proche de la vitesse désirée $\mathbf{v}_t^{\text{des}}$.

Chaque robot utilise des observations locales :

- Une observation proprioceptive $\mathbf{o}_{self}$ composée de sa vitesse $\mathbf{v}_t^i$, de son orientation ori , de sa vitesse désirée $\mathbf{v}_t^{\text{des}}$ et de son rayon de sécurité R_c .
- Une observation extéroceptive $\mathbf{o}_{sur}$ composée, pour chaque voisin j , d'un vecteur RVO $\mathbf{c}_j$, de la distance d_j et d'un indicateur de risque de collision r_j .

Ces informations sont utilisées en entrée d'un réseau de neurones qui apprend une politique π_θ , produisant une action sous la forme d'un incrément de vitesse $\Delta\mathbf{v}_t$. Cette politique est partagée par l'ensemble des robots.

Le problème revient alors à résoudre, pour chaque robot et à chaque pas de temps Δt , le problème d'optimisation contraint suivant :

$$\begin{aligned} & \arg \min_{\pi_\theta} \|\mathbf{v}_t - \mathbf{v}_t^{\text{des}}\| \\ \text{s.t. } & \Delta\mathbf{v}_t \sim \pi_\theta(\mathbf{a}_t \mid \mathbf{o}_{self}, \mathbf{o}_{sur}), \quad \mathbf{v}_t = \mathbf{v}_{t-1} + \mu \Delta\mathbf{v}_t, \\ & \mathbf{p}_t = \mathbf{p}_{t-1} + \Delta t \mathbf{v}_t, \quad d_j = \|\mathbf{p}_t - \mathbf{p}_{jt}\|, \quad \forall j \in [1, m], \quad d_j > R_c + R_{jc}. \end{aligned}$$

où $\|\cdot\|$ désigne la norme euclidienne et μ est un hyper-paramètre contrôlant l'amplitude de l'incrément de vitesse.

DRL : L'algorithme d'apprentissage par renforcement actor-critic repose sur un actor network qui génère les actions de l'agent à partir de ses observation, et un critic network qui évalue ces actions en estimation la fonction de valeur.

- **Espace des observations $\mathbf{o}$.** L'ensemble des observations correspond aux mesures proprioceptive et extéroceptives :

$$\mathbf{o} = [\mathbf{o}_{self}, \mathbf{o}_{sur}^j], \quad j = 0, \dots, m.$$

- **Espace des actions $\mathbf{a}$.** L'espace des actions correspond aux incréments vitesse sur le plan (x, y) .

$$\mathbf{a} = [\Delta v_x, \Delta v_y], \quad \text{où } \mathbf{v}_t \in [\mathbf{v}_{\min}, \mathbf{v}_{\max}].$$

Dans le cas de robots non-holonomes, cette vitesse est ensuite convertie en vitesses linéaire et angulaire.

- **Fonction de récompense r_{rvo} .** La fonction de récompense RVO évalue la qualité de la vitesse choisie en fonction du risque de collision estimé via les zones RVO. Elle est définie à chaque temps t :

$$r_{\text{rvo}}^t = \begin{cases} a - b \times \text{diff}_v & \text{si } \mathbf{v}_t \notin \text{RVO or } \xi > 5, \\ c - d \times (\xi + f)^{-1} & \text{si } \mathbf{v}_t \in \text{RVO and } \xi > 0.1, \\ -e \times (\xi + f)^{-1} & \text{si } \xi \leq 0.1. \end{cases}$$

où $\text{diff}_v = \|\mathbf{v}_t - \mathbf{v}_t^{\text{des}}\|$ et ξ correspond au temps minimal estimé avant une collision.

Les paramètres a, b, c, d, e et f sont des constantes ajustables. Les termes a et c récompensent les situations sûres ou proches de la vitesse désirée, tandis que b pondère l'importance du suivi de la vitesse désirée, et d et e pondèrent l'importance de l'évitement des collisions.

- **Condition de fin.** Un épisode prend fin dans 3 cas : tous les robots ont atteint la position cible, une collision survient ou l'épisode a atteint son nombre de steps maximal.

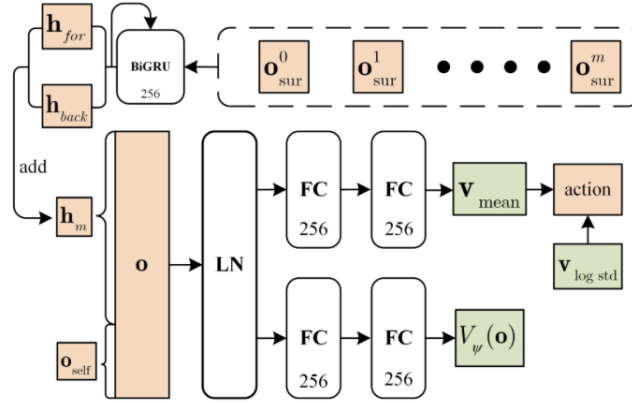


FIGURE 8 – Architecture du réseau de neurones BiGRU de la politique de navigation actor-critic. Source : [10]

Les observations extéroceptives sont de taille variable selon le voisinage du robot. Elles sont donc transformées en un vecteur de taille fixe à l'aide d'un réseau de type BiGRU, qui traite les informations dans les deux sens afin de capturer au mieux les interactions entre les agents. Le vecteur obtenu est ensuite concaténé avec les observations proprioceptives, et utilisé comme entrée dans le réseau de l'acteur, composé de 2 couches FC. Il produit l'incrément de vitesse, modélisé comme une distribution gaussienne. Le réseau critic utilise une architecture similaire, afin de produire un scalaire qui évalue l'état et l'action associée.

L'apprentissage est réalisé avec l'algorithme de proximal policy optimization (PPO).

ENTRAÎNEMENT

Les auteurs ont entraîné le modèle sur le circle scenario, dans lequel les robots sont initialement de manière uniforme répartis sur un cercle, avec des orientations aléatoires. Chaque robot doit atteindre un objectif situé à l'opposé du cercle, ce qui génère de nombreuses interactions et situations de collisions potentielles.

Afin d'améliorer la convergence de l'apprentissage, l'entraînement se déroule en 2 phases : (i) une phase simple avec peu de robots (2 à 4), afin d'apprendre les comportements de bases, puis (ii) une phase plus complexe avec plus de robots (10), afin d'introduire des interactions plus denses.

Détails : 10h d'entraînement – 200 epochs pour la phase simple – 1 000 epochs pour la phase complexe.

Setup : CPU i7-9700 et GPU Nvidia GTX 1080.

RÉSULTATS

Pour valider les performances du modèles, celui-ci a été comparé aux approches SARL, GA3C-CADRL et NG-ORCA dans des scénarios circulaires et aléatoires. Cette comparaison repose sur 3 critères : le taux de succès, le temps de trajet et la vitesse moyenne.

Pour ces trois métriques, RL-RVO présente de meilleures performances : le taux de succès est plus élevé, les temps de trajet sont plus courts, et les vitesses moyennes plus importantes. De plus, cette méthode est moins coûteuse en calcul que les autres, et présente une meilleure stabilité, avec un écart-type plus faible. Dans tous les cas, les performances diminuent lorsque le nombre de robots augmente (de 6 à 20 agents).

L'approche a également été évaluée en conditions réelles et comparée à la méthode NH-ORCA. Encore une fois, RL-RVO présente de meilleures performances en termes de robustesse et de taux de succès.

CONCLUSION

Dans le cadre de notre étude, on s'intéresse au problème de collision avoidance entre plusieurs AGV, en supposant l'absence de communication explicite entre les agents et une prise de décision basée sur des observations locales (pas de méta-agent).

Plus précisément, l'objectif est d'apprendre aux AGV à réguler leur vitesse pour éviter les obstacles (statiques et dynamiques), tout en minimisant leur temps de mission.

L'approche RL-RVO apparaît assez pertinente car elle correspond à un cadre entièrement décentralisé, sans planification globale de trajectoire. La faisabilité de cette méthode au sein du projet dépend toutefois de la structure de nos robots, notamment de leur capacité à estimer de manière fiable la vitesse des agents voisins à partir des capteurs embarqués.

De plus, la notion de répartition de l'effort d'évitement entre agents est assez intéressante d'un point de vue théorique, mais elle peut ne pas être adaptée pour des environnement très contraints, comme des circuits ou des couloirs étroits.

Ainsi, cette approche constitue un cadre prometteur pour la navigation de robots autonomes en environnement complexe, reposant uniquement sur des capteurs embarqués. Elle pourrait être envisagée dans un premier temps, où l'environnement est contrôlable, mais semble moins adaptée à notre objectif final de circulation sans collision des AGV sur un circuit, dans un contexte aéroportuaire.

[11] Decentralized non-communicating multiagent collision avoidance with deep reinforcement learning, Chen & al. (2017)

SUJET

Ce papier traite du problème de navigation sans collision, dans un système multi-agent décentralisé, où les robots ne communiquent pas entre eux. Leur solution repose sur du deep reinforcement learning (DRL), avec un apprentissage offline d'une fonction de valeur et une exécution online rapide pour choisir une vitesse.

MODÉLISATION DU PROBLÈME

Dans le cadre de l'apprentissage par renforcement, les auteurs formulent le problème comme un processus de décision de Markov (MDP), défini par le tuple $\langle S, A, P, R, \gamma \rangle$, où S est l'ensemble des états, A est l'ensemble des actions, P est la probabilité de transition entre les états, R est la fonction de récompense et γ est le facteur de discount. Dans un premier cas, le papier considère le cas où il n'y a que deux agents.

- **Espace des états S .** L'état du système correspond à la concaténation de l'état complet de l'agent considéré $\mathbf{s}$ et de la partie observable de l'état de l'autre agent $\tilde{\mathbf{s}}^o$:

$$\mathbf{s}^{jn} = [\mathbf{s}, \tilde{\mathbf{s}}^o] \in \mathbb{R}^{14}, \quad \mathbf{s} = [\mathbf{s}^o, \mathbf{s}^h] \in \mathbb{R}^9.$$

La partie observable correspond à des informations accessibles à tous les agents : la position $\mathbf{p}$, la vitesse $\mathbf{v}$ et le rayon de l'agent r . La partie cachée contient des informations internes à l'agent : sa position cible $\mathbf{p}_g$, sa vitesse désirée v_{pref} et son angle de direction θ . Plus particulièrement :

$$\mathbf{s}^o = [p_x, p_y, v_x, v_y, r] \in \mathbb{R}^5, \quad \mathbf{s}^h = [p_{gx}, p_{gy}, v_{pref}, \theta] \in \mathbb{R}^4.$$

- **Espace des actions A .** L'espace des actions correspond à l'ensemble des vecteurs de vitesses admissibles,

$$\mathbf{a} = \mathbf{v} \in \mathbb{R}^2, \quad \text{avec } \|\mathbf{v}\| \leq v_{pref}.$$

D'autres contraintes cinématiques seront ensuite intégrées, en restreignant davantage les vitesses et les directions admissibles, afin de garantir des actions physiquement réalisables par le robot.

- **Fonction de récompense R .** La fonction récompense un agent lorsqu'il atteint son objectif, et le pénalise lorsqu'il est trop proche de l'autre, en fonction de la distance de séparation minimum d_{min} ,

$$R(\mathbf{s}^{jn}, \mathbf{a}) = \begin{cases} -0.25 & \text{si } d_{min} < 0, \\ -0.1 - d_{min}/2 & \text{si } d_{min} < 0.2, \\ 1 & \text{si } \mathbf{p} = \mathbf{p}_g, \\ 0 & \text{sinon.} \end{cases}$$

- **Probabilité de transition P .** la dynamique du système est définie par les équations de mouvement des agents. Toutefois, la transition est partiellement inconnue car elle dépend de la politique et des intentions cachées de l'autre agents.

CADRL : L'objectif est donc de trouver la fonction de valeur optimale :

$$V^*(\mathbf{s}_0^{jn}) = \sum_{t=0}^T \gamma^{t \times v_{pref}} \times R(\mathbf{s}_t^{jn}, \pi^*(\mathbf{s}_t^{jn})),$$

à partir de laquelle on peut dériver la politique optimale :

$$\pi^*(\mathbf{s}_0^{jn}) = \arg \max_{\mathbf{a}} R(\mathbf{s}_0, \mathbf{a}) + \gamma^{t \times v_{pref}} \int_{\mathbf{s}_1^{jn}} P(\mathbf{s}_0^{jn}, \mathbf{s}_1^{jn}) V^*(\mathbf{s}_1^{jn}) d\mathbf{s}_1^{jn}.$$

Cette fonction de valeur est apprise à l'aide d'un réseau de neurones profond, qui permet d'estimer le coût à atteindre l'objectif à partir d'un état donné. Pour améliorer l'efficacité et la stabilité du modèle, les états sont exprimés dans un repère centré sur l'agent. Durant l'exécution, la politique est obtenue par une stratégie one-step look ahead : pour chaque action candidate, l'agent prédit l'état suivant (en supposant un déplacement constant des autres agents), puis sélectionne l'action qui minimise la somme de la récompense immédiate et de la valeur future estimée.

L'extension multi-agent repose sur une décomposition du problème en interactions pair-à-pair : pour chaque agent voisin, une valeur est estimée à l'aide du modèle appris en deux agents, puis l'action est choisie en optimisant le pire cas parmi les voisins. Cette approche approxime la dynamique globale, au lieu de modéliser le système multi-agent complet.

ENTRAÎNEMENT

L'apprentissage du modèle se déroule en 2 étapes :

1. La fonction de valeur est initialisée par apprentissage supervisé, avec des trajectoires générées par une méthode classique ORCA. Chaque trajectoire est transformée en un ensemble de paires état-valeur, où la valeur correspond au temps nécessaire pour atteindre l'objectif. Le réseau est entraîné par backpropagation, en minimisant l'erreur quadratique.
2. L'estimation est affinée par apprentissage par renforcement : des scénarios multi-agents sont générés, et la fonction de valeur est mise à jour à partir des trajectoires obtenues. Une stratégie ϵ -greedy et un replay buffer sont utilisés pour favoriser l'exploration et stabiliser l'apprentissage. Les auteurs introduisent également un terme de pénalisation pour décourager les comportements égoïstes (par exemple, un agent avance directement vers sa cible, en forçant les autres à faire de grands détours).

Détails : < 3h d'entraînement - 500 trajectoires avec 20 000 paires état-valeur durant la phase 1 – convergence au bout de 800 épisodes durant la phase 2.

Setup : i7-5820K CPU.

RÉSULTATS

Les performances du modèle sont comparées à la méthode classique ORCA, à partir d'une mesure du temps supplémentaire nécessaire pour atteindre l'objectif par rapport à une trajectoire en ligne droite. CADRL présente de meilleures performances que ORCA sur plusieurs configurations, avec un retard moyen plus faible, en particulier pour des scénarios complexes. Pour tous les modèles, les performances ainsi que leur stabilité se dégradent lorsque le nombre d'agents augmente (de 2 à 8), mais la supériorité de CADRL devient plus marquée.

La méthode reste efficace même en présence d'agents non-coopératifs ou d'obstacles statiques. Toutefois, elle peut échouer dans des environnements fortement contraints, où certains agents peuvent être bloqués.

CONCLUSION

Dans le cadre de notre étude, on s'intéresse au problème de collision avoidance entre plusieurs AGV, en supposant l'absence de communication explicite entre les agents et une prise de décision basée sur des observations locales (pas de méta-agent).

Plus précisément, l'objectif est d'apprendre aux AGV à réguler leur vitesse pour éviter les obstacles (statiques et dynamiques), tout en minimisant leur temps de mission.

CADRL repose sur la combinaison d'apprentissage supervisé et par renforcement, ce qui rend la méthode conceptuellement assez simple et efficace en pratique. Toutefois, plusieurs limitations peuvent être

relevées au regard de notre problématique : (i) la méthode présente des performances dégradées dans des environnements fortement contraints, tel qu'un circuit d'aéroport, (ii) l'observation complète des positions et vitesses des autres agents paraît peu réaliste, et (iii) la méthode est dite multi-agent, mais repose en réalité sur une décomposition pair-à-pair du problème, limitant la modélisation des interactions globales. Par ailleurs, si la méthode est plus scalable que ORCA, son passage à l'échelle reste tout de même approximatif.

Ainsi, cette approche peut être une base intéressante pour notre étude, notamment dans une perspective de combiner apprentissage supervisé à l'aide de méthodes classiques et apprentissage par renforcement. Cependant, la modélisation du problème et les hypothèses associées doivent être adaptées afin de mieux correspondre à notre cadre.

[12] Distributed Non-Communicating Multi-Robot Collision Avoidance via Map-Based Deep Reinforcement Learning, Chen & al. (2020)

SUJET

Ce papier traite du problème de navigation sans collision, dans un système multi-agent décentralisé, où les robots ne communiquent pas entre eux. Les auteurs proposent une approche de deep reinforcement learning (DRL) basée sur des cartes locales égocentriques pour représenter l'environnement autour de chaque robot.

Cette représentation permet de combiner les avantages des approches agent-level, précises mais coûteuses en perception, et celles sensor-level, plus simples mais limitées à des capteurs spécifiques. Les principales contributions sont les suivantes :

- Une méthode décentralisée et sans communication, qui utilise des cartes locales égocentriques.
- L'apprentissage d'une politique d'évitement de collision, qui utilise l'algorithme distributed proximal policy optimisation (DPPO), permettant de passer des cartes locales aux commandes du robot, amélioré avec une stratégie de curriculum learning.
- La validation en simulation et sur robots réels, avec de meilleures performances que les approches DRL existantes.
- Une étude d'ablation mettant en évidence les impacts positifs des cartes locales égocentriques et du curriculum learning.

MODÉLISATION DU PROBLÈME

Le problème de collision avoidance multi-agent est formulé comme un Partially Observable Markov Decision Process (POMDP), représenté par le tuple $\langle S, A, P, R, \Omega, O \rangle$, où S est l'ensemble des états, A est l'ensemble des actions, P est la probabilité de transition entre les états, R est la fonction de récompense, Ω est l'espace des observations et O est la distribution de l'observation selon l'état du système.

- **Espace des observations Ω .** L'observation d'un agent $\mathbf{o}_t$ est composée de sa carte locale égocentrique $\mathbf{M}_t$, sa position relative à l'objectif $\mathbf{g}_t$ et son angle d'orientation α_t :

$$\mathbf{o}_t = (\mathbf{M}_t, \mathbf{g}_t, \alpha_t).$$

La carte est construite à partir de costmaps, pouvant être générées via différents capteurs. Afin d'intégrer l'information temporelles, les trois dernières observations sont utilisées en entrée du réseau :

$$\vec{\mathbf{o}}_t = (\mathbf{o}_{t-2}, \mathbf{o}_{t-1}, \mathbf{o}_t).$$

- **Espace des actions A .** L'espace des actions est continu et correspond aux vitesses admissibles par l'agent, composée d'une vitesse linéaire v_t et d'une vitesse angulaire ω_t ,

$$\mathbf{a}_t = (v_t, \omega_t),$$

avec $v_t \in [0, 0.6]$ et $\omega_t \in [-0.9, 0.9]$.

- **Fonction de récompense R .** La fonction de récompense est définie comme la somme de 4 composantes :

$$r_t = r_t^g + r_t^a + r_t^c + r_t^s,$$

où r_t^g encourage la progression vers l'objectif, r_t^a récompense l'atteinte de la cible, r_t^c pénalise les collisions et r_t^s applique une pénalité à chaque étape afin d'encourager des trajectoires courtes.

- **Condition de fin.** Un épisode prend fin dans 3 cas : tous les robots ont atteint la position cible, une collision survient ou l'épisode a atteint son nombre de steps maximal.

DPPO : L'objectif est d'apprendre une politique qui maximise le retour cumulé, ce qui revient à minimiser le temps d'arrivée moyen de chaque robot tout en évitant les collisions. Chaque robot partage la même politique stochastique $\pi_\theta(\mathbf{a}_t | \vec{\mathbf{o}}_t)$, qui associe une action aux observations.

Soit τ une trajectoire, on souhaite maximiser l'espérance du retour cumulé :

$$J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)], \quad \text{avec} \quad R(\tau) = \sum_{t=0}^{+\infty} \gamma^t r_t.$$

Le gradient de la politique s'écrit :

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(\mathbf{a}_t | \vec{\mathbf{o}}_t) R(\tau) \right].$$

Afin de réduire la variance et de mieux évaluer la qualité des actions, on introduit une estimation de la fonction avantage $\hat{A}^{\pi_\theta}$, obtenue par Generalized Advantage Estimation (GAE) :

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(\mathbf{a}_t | \vec{\mathbf{o}}_t) \hat{A}^{\pi_\theta}(\vec{\mathbf{o}}_t, \mathbf{a}_t) \right].$$

DPPO repose sur une variante de PPO qui contraint les mises à jour de la politique afin d'assurer une meilleure stabilité de l'apprentissage, en limitant les variations trop importantes entre deux itérations. Plus particulièrement, l'objectif PPO est :

$$L^{PPO}(\theta) = \mathbb{E}_{\vec{\mathbf{o}}, \mathbf{a} \sim \pi_{\theta_k}} \left[\min \left(\frac{\pi_\theta(\mathbf{a} | \vec{\mathbf{o}})}{\pi_{\theta_k}(\mathbf{a} | \vec{\mathbf{o}})} \hat{A}^{\pi_{\theta_k}}(\vec{\mathbf{o}}, \mathbf{a}); \text{clipped} \left(\frac{\pi_\theta(\mathbf{a} | \vec{\mathbf{o}})}{\pi_{\theta_k}(\mathbf{a} | \vec{\mathbf{o}})} \hat{A}^{\pi_{\theta_k}}(\vec{\mathbf{o}}, \mathbf{a}) \right) \right) \right],$$

où ϵ est le ratio du clip, qui force la valeur à être entre $1 - \epsilon$ et $1 + \epsilon$. Les paramètres sont mis à jour par ascension de gradient :

$$\theta_{k+1} = \arg \max_{\theta} L^{PPO}(\theta).$$

Enfin, DPPO étend PPO à un cadre distribué, où plusieurs robots collectent des expériences en parallèle dans différents environnements, qui sont ensuite utilisées pour mettre à jour une politique partagée.

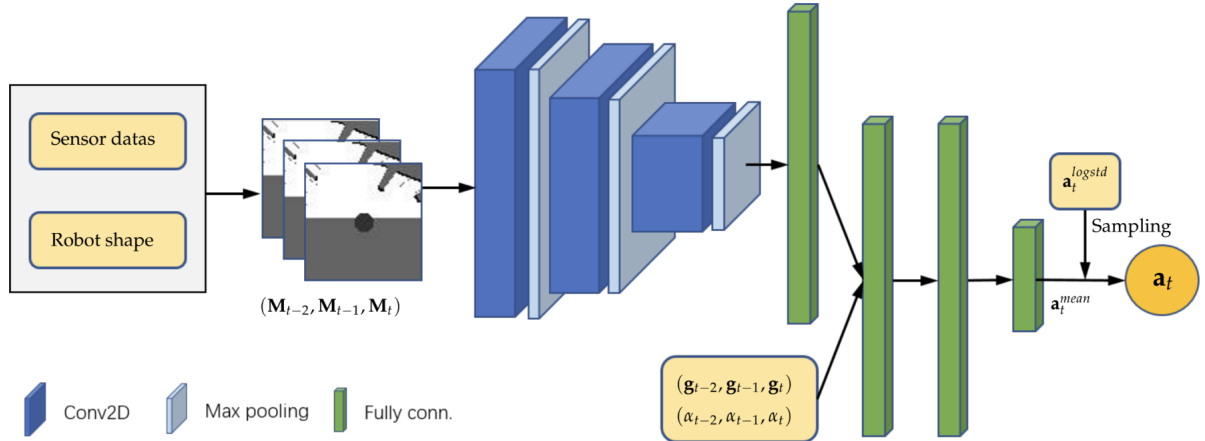


FIGURE 9 – Architecture du policy network. Source : [12]

Le modèle est composé d'un policy network et d'un value network, entraînés séparément avec des paramètres distincts, notés π_θ et V_ϕ .

- Le policy network prend en entrée $\vec{\mathbf{o}}_t$, composés des trois dernières cartes locales égocentriques, des trois positions relatives à la cible ainsi que des trois derniers angles d'orientation du robot. Les cartes sont d'abord traitées par un CNN (3 couches Conv2D, 3 couches de max-pooling et 1 couche FC), puis concaténées aux autres entrées. L'ensemble est ensuite traité par 3 couches

FC pour produire la sortie : la vitesse moyenne de l'action $\mathbf{a}_t^{mean} = (v_t^{mean}, \omega_t^{mean})$. Finalement, l'action est échantillonnée selon une distribution gaussienne :

$$\mathbf{a}_t \sim \mathcal{N}(\mathbf{a}_t^{mean}, \mathbf{a}_t^{\log std}),$$

où $\mathbf{a}_t^{\log std}$ est un ensemble de paramètres appris durant l'entraînement.

- Le value network a globalement la même architecture, mais sa dernière couche est modifiée pour produire une estimation scalaire de la valeur d'état $V_\phi(\vec{o}_t)$.

ENTRAÎNEMENT

Les auteurs adoptent le paradigme centralized training, decentralized execution (CTDE) : la politique partagée est entraînée de manière centralisée, puis exécutée de manière décentralisée par chaque robot sans communication. Les données sont collectées en parallèle par plusieurs robots dans différents environnements, puis utilisées pour mettre à jour la politique et la fonction de valeur avec DPPO. Cela permet un apprentissage plus rapide et robuste.

Le processus est renforcé par une stratégie de curriculum learning. Le modèle est d'abord entraîné dans des environnements aléatoires avec peu de robots et d'obstacles, pour apprendre l'évitement d'obstacles, puis dans des environnements circulaires, avec plus d'interactions entre les robots.

Détails : 12h d'entraînement pour 900 itérations – Exécution en 2ms sur GPU pour 8 robots en simulation – 25ms en conditions réelles, avec 30ms pour la génération des cartes.

Le learning-rate est également diminué entre les deux phases (aléatoire puis circulaire) pour passer d'un apprentissage rapide et exploratoire à un ajustement plus fin et stable de la politique.

Setup : i7-9900 CPU et Nvidia Titan RTX GPU.

RÉSULTATS

Pour évaluer ses performances, la méthode est comparée aux approches NH-ORCA et sensor-level, selon le taux de succès, le temps supplémentaire, la distance supplémentaire et la vitesse linéaire moyenne. Sur l'ensemble des environnements testés, l'approche map-based au stage 2 a toujours le meilleur taux de succès, avec des missions abouties plus rapidement à une vitesse plus élevées, même si le chemin n'est pas toujours le plus court.

Le modèle est également exécuté sur des configurations non vues, telles que : un scénario avec un robot non coopératif, un scénario à grande échelle (200 robots et 200 obstacles), et un scénario hétérogène où les robots sont de taille et de formes différentes. Dans tous les cas, le modèle généralise bien sur les environnements non-vus sans nécessiter de fine-tuning.

CONCLUSION

Dans le cadre de notre étude, on s'intéresse au problème de collision avoidance entre plusieurs AGV, en supposant l'absence de communication explicite entre les agents et une prise de décision basée sur des observations locales (pas de méta-agent).

Plus précisément, l'objectif est d'apprendre aux AGV à réguler leur vitesse pour éviter les obstacles (statiques et dynamiques), tout en minimisant leur temps de mission.

Globalement, l'approche semble particulièrement efficace dans des environnements denses qui comportent de nombreuses interactions, avec une bonne capacité de généralisation sur des scénarios non-vus. Elle s'appuie sur le paradigme CTDE, ce qui permet d'obtenir des résultats plus robustes en exploitant davantage d'informations durant l'entraînement, au prix toutefois d'un coût de calcul plus important.

De plus, contrairement aux méthodes vues précédemment qui utilisent un planificateur global, ici l'apprentissage est entièrement basé sur des observations locales. Cela apporte davantage de flexibilité, mais peut également complexifier l'apprentissage et nécessiter un temps de calcul plus important.

Par ailleurs, la stratégie de curriculum learning semble assez pertinente pour stabiliser et accélérer l'apprentissage, et sera intégrée à notre étude.

[13] Attention Graph for Multi-Robot Social Navigation with Deep Reinforcement Learning, Escudie & al. (2024)

SUJET

Ce papier traite du problème de navigation sociale multi-robots sans collision, dans des environnements denses, où les robots naviguent parmi des humains, sans communication explicite. Les auteurs proposent MultiSoc, une approche basée sur une architecture Graph Neural Network (GNN) qui permet de traiter les interactions humano-robot et robot-robot.

Les principales contributions sont les suivantes :

- Une modélisation par graphes des interactions pour la navigation sociale multi-robots/
- L'introduction d'un méta-paramètre permettant d'ajuster la densité du voisinage pris en compte.
- Une méthode capable d'apprendre efficacement la coordination implicite entre les robots, dans des foules denses et hétérogènes.
- Une meilleure généralisation et scalabilité par rapport aux approches mono-robot existantes.

MODÉLISATION DU PROBLÈME

MultiSoc peut être vu comme une sorte d'interpolation entre AttnGraph, dont il conserve le mécanisme d'attention ainsi que la prise en compte des positions futures prédites des entités, et de MAGE-X, dont il reprend une stratégie améliorée de sélection des arêtes ainsi qu'un mécanisme de fusion précoce des entités dans le graphe.

Pour chaque agent j , la pipeline est la suivante :

- Un Edge-Selector qui applique un mécanisme d'attention sur les nœuds correspondant aux entités visibles dans le champ de vision de l'agent. Cela produit un graphe dirigé et parcimonieux, dont la densité est contrôlée par le paramètre N_{head} .
- Un Crowd Coordinator, basé sur un Graph Attention Network (GAT) à une couche, qui agrège les informations des voisins pour chaque nœud. En parallèle, un Intrinsic Coordinator produit un résumé des contraintes liées à l'objectif du robot.
- Le nœud correspondant à l'agent j est ensuite extrait et concaténé avec la représentation issue de l'Intrinsic Coordinator.
- Un GRU, suivi de deux MLP, produit à la fois la valeur et l'action, conditionnellement à l'état caché précédemment et les informations agrégées.

De cette manière, l'utilisation de GNN permet la prise en compte d'un nombre variable d'entités, humaines ou robotiques, toutes intégrées dans un même graphe. La prise de décision est pseudo-centralisée, chaque agent infère les intentions des autres à partir de ses observations locales. Le champ réceptif de chaque agent est progressivement élargi, chaque couche de GNN augmentant la portée des interactions indirectes entre nœuds.

Pour chaque agent j , les données passées en entrée sont :

- Les trajectoires futures $T_i = [p_i^t, \dots, p_i^{t+5}]$ pour chaque entité $i = 1, \dots, N$, obtenues par une prédiction à vitesse constante sur un horizon de 5 pas de temps. La vitesse est estimée à partir de la différence entre les positions courante et précédente, où p_i^t est la position de l'entité i au temps t .
- Un graphe $G^j = (\mathbf{E}, \mathbf{M}^j)$, où $\mathbf{E}$ est l'ensemble des nœuds e_i , composés des trajectoires prédites T_i ainsi que d'un label (robot ou humain), et $\mathbf{M}^j \in \mathbb{R}^{N \times N}$ est la matrice d'adjacence (ou de visibilité) du point de vue de l'agent j , telle que :

$$M_{i,k}^j = \begin{cases} 1 & \text{si } i \text{ voit } k \text{ et } i \text{ est vu par } j, \\ 0 & \text{sinon.} \end{cases}$$

En parallèle, l'agent j dispose de ses informations intrinsèques $w^j = [p, v, g, \theta, r]$ où p est sa position actuelle, v sa vitesse, g son objectif, θ son orientation et r son rayon. L'état de l'agent est donc donné par :

$$s_t^j = [w^j, G^j].$$

Edge-Selector : L'objectif est de produire un graphe dirigé et parcimonieux, en ne conservant que les interactions les plus pertinentes entre entités. Plus précisément, un mécanisme de Multi-Head Attention (MHA) avec N_{head} têtes est appliqué sur les features des nœuds du graphe, afin de produire des scores $a_{i,j}^k$ représentant l'influence de l'entité j sur l'entité i pour chaque tête k . Afin de sélectionner le sous-ensemble d'arêtes pertinentes, un masque $\mathbf{M}_S$ est généré à l'aide du Gumbal-Softmax :

$$s_{i,j}^k = \text{Softmax}_j \frac{\text{MLP}(a_{i,j}^k) + g}{\tau}, \quad m_{i,j} = \frac{1}{N_{head}} \sum_{k=0}^{N_{head}} s_{i,j}^k,$$

où g est un bruit échantillonné selon une distribution de Gumbel, τ est un paramètre de température et MLP est une couche linéaire. Le graphe parcimonieux final $G_S = (\mathbf{E}_S, \mathbf{M}_S)$ est construit comme suit :

- Les features de chaque nœud correspondent aux scores d'attention pour l'agent i .
- Il y a une arête entre les nœuds i et j si $m_{i,j} \neq 0$. Chaque nœud a au plus N_{head} arêtes.

Crowd Coordinator : Une fois qu'on dispose du graphe parcimonieux, l'objectif est de propager l'information entre les nœuds voisins afin d'enrichir leurs représentations en fonction de leur voisinage. Pour cela, un mécanisme d'attention est appliqué au graphe parcimonieux, à partir duquel chaque tête d'attention k produit des coefficients $\alpha_{i,j}^k$ mesurant l'influence du nœud j sur le nœud i . Les nouvelles features sont ensuite obtenues par agrégation pondérée des voisins, et le nouveau graphe G_C est construit.

Constraints Coordinator : Le Crowd Coordinator a produit un graphe G_C dans lequel chaque nœud encode l'influence de son voisinage. Cependant, pour la prise de décision, seul le nœud correspondant à l'agent est nécessaire. Celui-ci est donc extrait puis concaténé avec la sortie du Intrinsic Coordinator, implémenté sous la forme d'un MLP. On obtient finalement un vecteur qui regroupe à la fois les contraintes externes liées aux autres entités (via le Crowd Coordinator), et la contrainte interne liée à l'objectif (via l'Intrinsic Coordinator). La cohérence temporelle dans les décisions est garantie à l'aide d'un Gated Recurrent Unit (GRU).

RL : Le modèle est alors modélisé comme un Multi-Agent Markov Decision Process (MAMDP). La fonction de récompense est construite de sorte à pénaliser l'agent s'il se trouve sur la trajectoire prédite d'une entité, tout en encourageant sa progression vers l'objectif. L'épisode s'achève lorsque tous les agents ont atteint leur objectif.

ENTRAÎNEMENT

Le modèle est entraîné avec l'algorithme Multi-Agent Proximal Policy Optimization (MAPPO), en suivant le paradigme Centralized Training, Decentralized Execution (CTDE).

Les auteurs étendent le simulateur mono-agent CrowdNav utilisé dans AttnGraph vers une version multi-agents, avec une amélioration de la gestion des matrices. Le simulateur multi-agent MultiCrowdNav repose sur des multi-particle environments (MPE), dans lesquels les agents doivent naviguer et communiquer, ce qui facilite le passage de PPO à MAPPO.

Les scénarios sont initialisés avec H humains disposés sur un cercle, chacun ayant pour objectif de rejoindre un point opposé, ce qui induit des trajectoires croisées. Lorsqu'un humain a atteint son objectif, un autre lui est attribué. Les humains interagissent entre eux, mais ignorent les robots. Ils sont soit contrôlés par ORCA, soit par Social Force, afin de modéliser des comportements hétérogènes.

Les R robots sont placés aléatoirement avec des objectifs attribués aléatoirement aussi. Lorsqu'un agent a atteint son objectif ou entre en collision, il continue de se déplacer mais n'est plus pris en compte dans les métriques, il correspond désormais à un obstacle dynamique pour les autres.

L'épisode se termine lorsque tous les agents sont soit entrés en collision, soit atteint leur objectif.

RÉSULTATS

Les performances sont évaluées selon les métriques suivantes : le taux de collision, le ratio d'intrusion, la durée et la distance moyennes des trajectoires, et la récompense moyenne à la fin de l'épisode.

Les auteurs comparent MultiSoc à AttnGraph dans plusieurs scénarios :

- 1 robot, 20 humains : Les deux modèles présentent des performances similaires.
- 5 robots, 20 humains : AttnGraph voit ses performances chuter (85% de succès), car il considère les autres robots comme des humains, alors que MultiSoc, entraîné sur un seul robot, montre de bonne capacité de généralisation (94% de succès).
- 6 robots, 6 humains : Les performances d'AttnGraph se dégradent fortement (68% de succès), alors que MultiSoc reste plus robuste, bien que la coordination soit plus complexe.

CONCLUSION

Dans le cadre de notre étude, on s'intéresse au problème de collision évitement entre plusieurs AGV, en supposant l'absence de communication explicite entre les agents et une prise de décision basée sur des observations locales (pas de méta-agent).

Plus précisément, l'objectif est d'apprendre aux AGV à réguler leur vitesse pour éviter les obstacles (statiques et dynamiques), tout en minimisant leur temps de mission.

L'approche MultiSoc propose une solution avancée pour ce type de problèmes, avec une modélisation fine des interactions entre agents. Elle permet notamment une coordination implicite efficace dans des environnements denses et hétérogènes.

Toutefois, cette méthode reste complexe à mettre en œuvre et est principalement conçue pour des scénarios de navigation sociale impliquant des humains. Dans le cadre de notre projet, une telle approche peut apparaître surdimensionnée dans un premier temps.

MultiSoc constitue une perspective intéressante si l'on souhaite étendre le système à des environnements plus complexes, notamment en présence de piétons ou d'agents aux comportements hétérogènes. Son utilisation pourrait ainsi être envisagée comme une extension future du projet si l'occasion se présente.

[14] Automated guided vehicle systems, state-of-the-art control algorithms and techniques, Deryck & al. (2020)

SUJET

Ce papier présente un état de l’art des systèmes de véhicules guidés automatisés (AGV) en se concentrant sur les algorithmes et techniques de contrôle. Plus particulièrement, il couvre les approches historiques et actuelles, ainsi que des méthodes émergentes prometteuses.

RÉSUMÉ

Les AGV sont des robots mobiles largement utilisés dans l’industrie pour le transport de marchandises. Dans les systèmes actuels, une flotte d’AGV est généralement organisée de manière centralisée : un meta-agent planifie les tâches et coordonne les autres. Cependant, cette architecture présente des limites en terme de communication, de mémoire et de capacité de calcul, en particulier lorsque la taille des systèmes augmente.

Dans ce contexte, la recherche actuelle s’oriente vers des méthodes décentralisées, dans lesquelles chaque agent prend des décisions localement tout en poursuivant un objectif global. Cette transition s’inscrit dans le cadre de l’industrie 4.0, qui nécessite des systèmes plus flexibles et robustes.

Les auteurs mettent en évidence les avantages et inconvénients des méthodes décentralisées, en soulignant leur capacité à s’adapter à des environnements dynamiques. Toutefois, elles nécessitent une coordination plus complexe entre agent, comparées aux méthodes centralisées, et ne garantissent pas un optimum global. Ainsi, elles impliquent toujours un compromis entre flexibilité et optimalité.

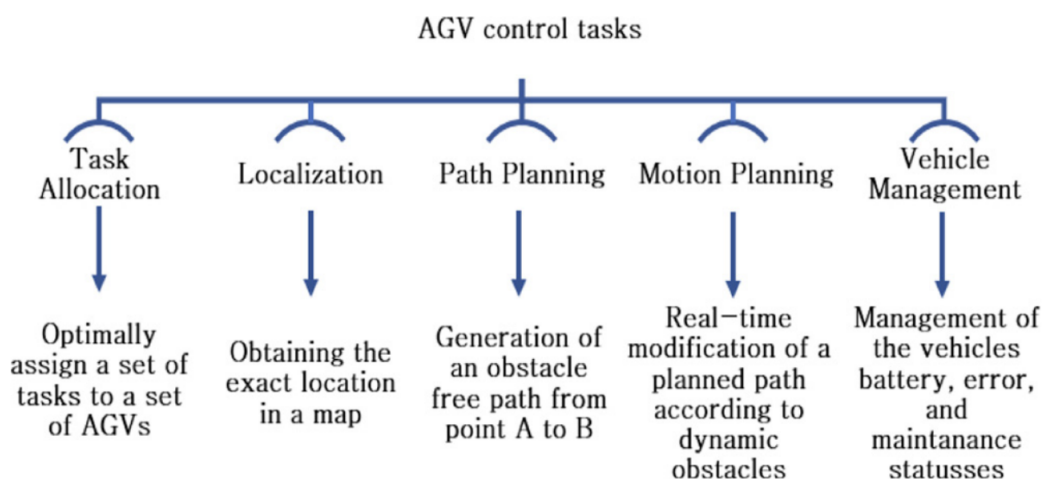


FIGURE 10 – Tâches de contrôle des AGV. Source : [14]

Le papier propose alors une décomposition du contrôle des AGV en 5 tâches principales (cf. Figure 10), qui sont ensuite analysées au regard des approches décentralisées. Ici, on s’intéressera principalement au motion planning : durant son trajet, l’AGV peut être confronté à des obstacles statiques et dynamiques, tels que des piétons ou d’autres AGV. Il est donc nécessaire d’éviter les collisions et les situations de blocage (deadlock).

La plupart des systèmes industriels reposent sur une approche de motion planning centralisée : un agent central planifie des trajectoires sans collision pour tous, en utilisant l’information globale. Bien que cette approche soit efficace, elle est limitée par des contraintes computationnelles, en particulier lorsque le nombre d’agents augmente. De plus, l’agent central constitue un point de défaillance unique.

Au contraire, dans les approches distribuées, chaque AGV suit un chemin prédéfini tout en décidant localement de la stratégie, afin de réagir aux perturbations. Cela améliore la robustesse et la scalabilité du système, tout en supprimant le point de défaillance unique.

Les auteurs décomposent le motion planning en 3 sous-problèmes : l'évitement de collision (collision avoidance), l'évitement de blocage (deadlock avoidance) et le contrôle de zones (zone control).

Dans la suite, on se concentre principalement sur le problème de collision avoidance. Le moyen le plus simple consiste à adapter la vitesse de l'AGV, en définissant une zone de sûreté autour de celui-ci. Pour cela, l'AGV doit être capable d'observer cette zone, à l'aide de caméras ou de capteurs. Certaines méthodes plus avancées consistent à éviter l'obstacle en le contournant, ou en planifiant un chemin alternatif.

Méthode	Avantages	Inconvénients
<i>Collision avoidance centralisée</i>		
Général	Optimal pour des petites flottes	Manque de robustesse, de scalabilité et de flexibilité pour de grandes flottes
<i>Collision avoidance décentralisée</i>		
Forward sensing	Simple, robuste et efficace comme vérification finale	–
Re-planning	Simple et rapide	Nécessite une carte globale de l'environnement
Déviations locale	Procédure de déviation sécuritaire	Adaptée à la navigation libre, difficultés avec des obstacles proches
VFH	Simple et efficace	Adaptée à la navigation libre
ORCA	Simple et efficace	Adaptée à la navigation libre
Modèles prédictifs	Performants en environnements denses	Complexes pour des cas simples, temps d'entraînement élevé

TABLE 1 – Comparaison des algorithmes de collision avoidance. Source : [14]

Collision avoidance centralisée : Un agent central utilise l'ensemble des informations (positions des AGV, objectifs et environnement) afin de générer des trajectoires sans collision pour l'ensemble des agents. Cependant, les positions des AGV changent continuellement, et ce processus doit être relancé fréquemment, ce qui est coûteux en temps de calcul et en ressources.

Les auteurs mettent en évidence 3 limites principales :

- L'augmentation du nombre d'agents rend ces calculs encore plus coûteux, rendant ces méthodes non scalables.
- Les temps de calculs sont élevés, ce qui introduit un décalage avec la situation réelle, limitant la réactivité en environnement dynamique.
- L'agent central ne prend en compte que les éléments connus de l'environnement. Par conséquent, il ne réagit pas efficacement aux obstacles imprévus ou dynamiques, tels que des piétons.

Par ailleurs, ces méthodes sont souvent intégrées à des problèmes d'allocation de tâches, ce qui conduit à des optimisations globales complexes. Ainsi, les auteurs suggèrent des approches décentralisées, dans lesquelles les AGV adaptent leur comportement à un environnement local. Cela permet une meilleure adaptation aux environnements dynamiques, tout en réduisant les besoins computationnels.

Collision avoidance décentralisée : L'AGV réagit en fonction de ce qu'il perçoit localement. En cas de collision potentiel, il peut soit adapter sa vitesse, ou contourner l'obstacle. Ces décisions reposent sur des capteurs embarqués, ou parfois, sur des échanges d'information avec les agents au voisinage.

- **Forward sensing.** L'AGV maintient une distance de sécurité avec les objets devant lui, grâce à des capteurs. Lorsqu'un obstacle entre dans cette zone, l'AGV ralentit jusqu'à s'arrêter si nécessaire, puis reprend sa trajectoire lorsque la voie est dégagée. Cette méthode constitue une solution simple, et largement utilisée comme sécurité de base dans les systèmes.

- **Re-planning.** L'AGV re-planifie sa trajectoire lorsqu'un obstacle imprévu est détecté. Cette re-planification repose sur une carte de l'environnement, qui contient l'ensemble des obstacles statiques et dynamiques.
- **Déviation locale.** L'AGV utilise une information locale issue des capteurs embarqués pour contourner un obstacle. La méthode se décompose en 4 étapes : vérification de sécurité, sortie de la trajectoire, contournement de l'obstacle puis retour à la trajectoire initiale. Les trajectoires générées sont des courbes polynomiales, similaires à des changements de voie, qui prennent en compte les contraintes cinématiques et dynamiques du robot. Cette méthode nécessite que le robot connaisse son circuit, et dispose de capteurs embarqués.
- **Virtual force fields (VFF).** L'environnement de l'AGV est représenté sous forme de grille d'occupation, où chaque case occupée génère une force répulsive inversement proportionnelle au carré de la distance au robot. Le déplacement de l'AGV dépend de la superposition de ces forces. Cette méthode repose sur une carte globale qui contient les obstacles statiques et dynamiques. Cependant, elle présente des limitations, notamment l'incapacité à naviguer entre deux objets rapprochés et la non prise en compte des contraintes cinématiques et dynamiques du robot.
- **Vector field histogram (VFH).** L'environnement de l'AGV est représenté comme un histogramme polaire centré sur le robot, qui décrit la densité d'obstacles selon différentes directions. L'action est choisie en minimisant cette densité. L'extension VFH+ améliore les trajectoires en prenant en compte la largeur du robot. Cette méthode nécessite une carte globale de l'environnement ou des capteurs embarqués, mais ne prend pas en compte les contraintes cinématiques et dynamiques du robot.
- **Dynamic window approach (DWA).** L'algorithme repose sur l'évaluation d'un ensemble de vitesses admissibles, qui prennent en compte les contraintes cinématiques et dynamiques du robot. A chaque intervalle de temps τ , les vitesses menant à des collisions ou non réalisables sont exclues. L'action est choisie en optimisant une fonction objectif, garantissant un déplacement sans collision à court terme. Cette méthode nécessite une carte globale de l'environnement ou des capteurs embarqués.
- **Optimal Reciprocal Collision Avoidance (ORCA).** Chaque AGV connaît sa position et sa vitesse, ainsi que celles des autres agents. De la même manière que DWA, l'algorithme repose sur l'évaluation d'un ensemble de vitesses admissibles, dont on exclut celles qui mènent à des collisions. L'action est choisie en optimisant un problème de programmation linéaire, qui consiste à sélectionner une commande sûre et proche de la vitesse désirée. Cette méthode prend en compte les contraintes cinématiques et dynamiques du robot.
- **Modèles prédictifs.** L'AGV utilise de l'apprentissage par renforcement profond pour sélectionner une action (vecteur de vitesse) à partir des observations de son environnement, en anticipant les mouvements des autres agents. L'apprentissage est guidé par un système de récompenses, et peut être basé sur des données générées par des méthodes classiques telles que ORCA.

Finalement, la collision avoidance centralisée est très coûteuses en calcul. Au contraire, l'approche décentralisée permet de réduire les ressources nécessaires, en adaptant le comportement de l'AGV à son environnement local. Ces trajectoires sans collisions peuvent être déterminées à partir de capteurs embarqués, d'une carte globale, ou des deux.

CONCLUSION

Dans le cadre de notre étude, on s'intéresse au problème de collision avoidance entre plusieurs AGV, en supposant l'absence de communication explicite entre les agents et une prise de décision basée sur des observations locales (pas de méta-agent).

Plus précisément, l'objectif est d'apprendre aux AGV à réguler leur vitesse pour éviter les obstacles (statiques et dynamiques), tout en minimisant leur temps de mission.

Les méthodes de re-planification sont particulièrement efficaces dans des environnements simples, où le

déplacement du robot est contraint (par exemple, sur un graphe). Cependant, elles agissent principalement sur les trajectoires. Or, notre approche vise à agir sur le contrôle de la vitesse.

Les approches VFH, DWA et ORCA s'inscrivent dans ce cadre, car elles peuvent fonctionner à partir d'informations locales issues des capteurs embarqués, sans nécessiter une connaissance de l'environnement globale.

Toutefois, notre projet s'oriente vers l'étude de méthodes d'apprentissage par renforcement, en particulier celles basées sur l'apprentissage profond. Ces méthodes permettent d'apprendre des stratégies de navigation complexes, et sont mieux adaptées aux contextes dynamiques et multi-agents, bien qu'elles nécessitent parfois un temps d'entraînement important.

[9] Towards optimally decentralized multi-robot collision avoidance via deep reinforcement learning, Long & al. (2018)

SUJET

Ce papier présente une méthode de collision avoidance multi-agents dans un cadre décentralisé.

Les approches centralisées sont certes performantes mais peu scalables et fortement dépendantes d'un serveur central et d'une communication fiable entre les robots. A l'inverse, les approches décentralisées sont plus robustes et scalables, mais elles restent souvent sensibles aux paramètres et présentent des performances inférieures.

Pour répondre à ces limites, les auteurs proposent une méthode qui permet d'apprendre une politique de navigation sensor-level, en transformant directement les données brutes issues des capteurs en commandes de vitesses, sans une estimation explicite des autres agents. Celle-ci repose sur du deep reinforcement learning, entraîné via une méthode de policy gradient.

MODÉLISATION DU PROBLÈME

Le problème de collision avoidance multi-robots est défini à partir de differential drive robots qui se déplacent sur un espace euclidien, contenant des obstacles ainsi que d'autres robots. Durant l'entraînement, il y a N robots, modélisés par des disques de rayon R .

A chaque unité de temps t , chaque robot i possède une observation o^t et prédit une action a^t à partir de celle-ci. Chaque observation correspond à une information partielle de l'état réel du système : $o^t \sim (s^t)$. Cela implique que le robot n'a pas accès à l'état des autres.

Chaque robot choisit l'action à réaliser de manière indépendante, échantillonnée à partir d'une politique stochastique partagée entre tous les agents,

$$a^t \sim \pi_\theta(a^t \mid o^t).$$

Cette action correspond à une vitesse v^t qui guide le robot vers son objectif en évitant les collisions avec les autres agents et les obstacles B_k pendant l'intervalle de temps t , jusqu'à l'observation suivante o^{t+1} . On note t_i^g , le temps de trajet du robot i , de sa position initiale à son objectif.

L'ensemble des trajectoires des robots est défini comme tel, soumis aux contraintes de cinématique et d'évitement de collision :

$$\mathbb{L} = \{l_i, i = 1, \dots, N \mid v_i^t \sim \pi_\theta(a_i^t \mid o_i^t), p_i^t = p_i^{t-1} + \Delta t \times v_i^t, C_i\},$$

où

$$C_i = \left\{ \forall j = 1, \dots, N, j \neq i, \|p_i^t - p_j^t\| > 2R \wedge \forall k = 1, \dots, M, \|p_i^t - B_k\| > R \wedge \|v_i^t\| > v_i^{max} \right\}.$$

L'objectif est donc de minimiser le temps moyen d'arrivée des robots :

$$\arg \min_{\pi_\theta} \mathbb{E} \left[\frac{1}{N} \sum_{i=1}^N t_i^g \mid \pi_\theta \right].$$

Le problème est formulé comme un POMDP, représenté par le tuple (S, A, P, R, Ω, O) où S est l'ensemble des états, A est l'ensemble des actions, P est la probabilité de transition entre les états, R est la fonction de récompense, Ω est l'espace des observations et O est la distribution de l'observation selon l'état du système.

- **Espace des observations O .** L'espace de o^t est composée de 3 éléments :

$$o^t = [o_z^t, o_g^t, o_v^t]$$

où $o_z^t \in \mathbb{R}^{3 \times 512}$ correspond aux mesures laser 2D de l'environnement local, $o_g^t \in \mathbb{R}^2$ correspond à la position relative (distance, angle) de l'objectif dans le repère du robot et $o_v^t \in \mathbb{R}^2$ correspond à la vitesse (linéaire et angulaire) actuelle du robot.

- **Espace des actions A .** L'ensemble des actions correspond aux vitesses autorisées sur un espace continu, définies par les vitesses de translation et de rotation du robot,

$$a^t = v^t = [\nu^t, \omega^t], \quad \text{où } \nu \in]0, 1[, \omega \in]-1, 1[.$$

- **Fonction de récompense R .** L'objectif est de minimiser le temps d'arriver moyen de tous les robots, tout en évitant les collisions. On a alors :

$$r^t = g_r^t + c_r^t + w_r^t,$$

où g_r récompense le robot lorsqu'il se rapproche et atteint son objectif, c_r pénalise le robot lorsqu'il entre en collision et w_r encourage le robot à se déplacer de manière fluide.

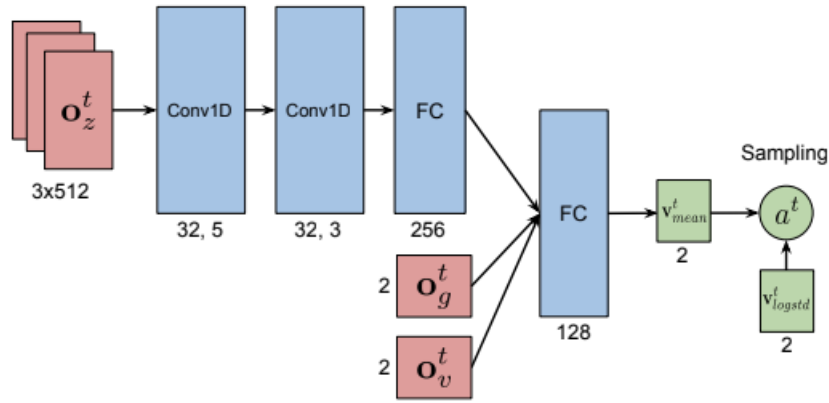


FIGURE 11 – Architecture du réseau de neurones pour la collision avoidance. Source : [9]

Les auteurs ont construit un réseau de neurones à 4 couches pour approximer la politique π_θ , prenant en entrée l'observation o^t et produisant l'action a^t sous forme de vitesse v^t en sortie. Les 3 premières couches (2 couches de convolution 1D + 1 couche fully-connected) sont dédiées au traitement des données o_z^t issues du capteur laser. La représentation obtenue est concaténée aux deux autres composantes de l'observation, o_g^t et o_v^t , puis passée dans 1 couche fully-connected. La sortie est constituée de la moyennes de l'action v_μ^t , où une fonction sigmoïde est utilisée pour contraindre la vitesse linéaire, et une fonction tanh est utilisée pour limiter la vitesse angulaire. Finalement, l'action finale est échantillonnée à partir d'une distribution gaussienne :

$$a^t \sim \mathcal{N}(v_\mu^t, v_{\log \sigma}^t)$$

où v_μ^t représente la moyenne prédite par le réseau et $v_{\log \sigma}^t$ est un vecteur de log-écart-type appris durant la phase d'entraînement.

Les auteurs proposent une extension de l'algorithme de proximal policy optimization (PPO), qui suit le paradigme centralized training and decentralized execution (CTDE). Chaque robot exécute localement la politique partagée π_θ à partir de ses propres observations, tandis que l'entraînement est réalisé à partir des expériences collectées en parallèle par tous les robots.

Le processus d'apprentissage alterne entre deux phases :

- La collecte de T_{max} données de trajectoires en exécutant la politique en parallèle.
- La mise à jour de la politique en utilisant ces données pour construire la perte surrogée $L_{PPO}(\theta)$, optimisée avec Adam sous contrainte de divergence de KL.

En parallèle, un deuxième réseau de neurones approxime la fonction de valeur $V_\phi(s^t)$ pour estimer l'avantage A^t . Son architecture est similaire à celle du réseau de politique, mais avec une seule sortie linéaire. Il est entraîné via une perte quadratique $L_V(\phi)$, optimisée avec Adam.

Les paramètres de la politique π_θ et de la fonction de valeur V_ϕ sont appris séparément, ce qui mène à de meilleurs résultats en pratique.

ENTRAÎNEMENT

Afin d'exposer les robots à une grande diversité d'environnements, les auteurs conçoivent plusieurs scénarios incluant différents types d'obstacles, à l'aide du Stage mobile robot simulator. Dans la majorité des scénarios, les zones de départ et d'arrivée sont définies dans l'espace de travail, puis les positions initiales et les objectifs des robots sont échantillonnées aléatoirement. La diversité et la complexité des scénarios permet aux robots d'améliorer la qualité et la robustesse de la politique apprise. Combinée au paradigme CTDE, cette approche permet d'optimiser efficacement la politique de collision avoidance sur une grande variété d'environnement.

L'entraînement sur plusieurs environnements simultanément permet d'avoir des performances plus robustes mais rend cependant l'apprentissage plus compliqué. Pour pallier cela, les auteurs proposent un processus d'entraînement étapes, inspiré du curriculum learning, pour accélérer la convergence de la politique vers une solution satisfaisante. L'idée consiste à commencer l'entraînement avec peu de robots sur des scénarios simples, et de peu à peu augmenter le nombre de robots et la complexité des scénarios.

Détails : 12h d'entraînement – Exécution = 3ms sur CPU, 1.3ms sur GPU pour 10 robots.

Setup : i7-7700 CPU – Nvidia GTX 1080 GPU.

RÉSULTATS

Chaque scénario a été testé 50 fois et évalué selon les métriques suivantes : le success rate, l'extra time (différence entre le temps de trajet moyen et un temps minimal théorique), l'extra distance (différence entre la distance parcourue et la distance minimale), et la vitesse moyenne.

Plusieurs types de scénarios (circulaire, aléatoire, et de groupe) ont été comparés à des méthodes classiques comme NH-ORCA ou une politique apprise par supervision (SL-policy). Les résultats montrent une amélioration significative du taux de succès et une réduction du temps de trajets. Bien que les trajectoires soient parfois plus longues, cela est compensé par une vitesse moyenne plus élevée.

La politique finale (Stage 2) montre de meilleures performances que celle entraînée uniquement en première phase (Stage 1), qui tend à sur-apprendre certains environnements.

Les expériences mettent aussi en évidence une forte capacité de généralisation : la politique apprise fonctionne efficacement dans des situations non vues durant l'entraînement, y compris dans des environnements complexes et à grande échelle.

CONCLUSION

Dans le cadre de notre étude, on s'intéresse au problème de collision avoidance entre plusieurs AGV, en supposant l'absence de communication explicite entre les agents et une prise de décision basée sur des observations locales (pas de méta-agent).

Plus précisément, l'objectif est d'apprendre aux AGV à réguler leur vitesse pour éviter les obstacles

(statiques et dynamiques), tout en minimisant leur temps de mission. Ce papier correspond à notre cadre : les autres agents sont modélisés comme des obstacles dynamiques, et chaque AGV régule leur vitesse à partir d'observations partielles, pour éviter les collisions.

Cette méthode pourrait être appliquée à notre problème avec certaines adaptations, notamment concernant la modélisation de la dynamique des AGV.

L'approche CTDE est particulièrement pertinente dans le cadre multi-agents sans communication : elle permet d'apprendre une politique robuste à partir des expériences collectives, tout en conservant une exécution individuelle.

Cependant, l'algorithme PPO est une méthode on-policy, ce qui peut limiter l'efficacité de l'apprentissage. Bien que les auteurs déclarent un temps d'entraînement de 12h, cela dépend du cadre expérimental et de la complexité des environnements.

Par ailleurs, l'approche repose uniquement sur des observations locales, sans planification globale explicite, ce qui peut conduire à des comportements sous-optimaux ou des goulots d'entraînement.

Finalement, cette méthode semble prometteuse, notamment pour sa capacité de généralisation et sa robustesse en environnement multi-agents.

SUJET

La vidéo est une retranscription d'un cours sur le Multi-Agent Reinforcement Learning (MARL), animé par Stefano V. Albrecht et organisé par Artificial Intelligence Research Institute (IAAA-CSIC). On s'intéresse ici à l'extrait qui présente le paradigme Centralized Training, Decentralized Execution (CTDE).

RÉSUMÉ

En MARL, on distingue deux phases : l'entraînement et l'exécution. La phase d'entraînement correspond à l'apprentissage et l'optimisation des paramètres du modèle, alors que la phase d'exécution correspond à son utilisation, sans modification des paramètres. L'information disponible diffère durant ces deux phases.

Dans le cadre de l'Independent Learning (IL), chaque agent considère les autres comme faisant partie de l'environnement. Il ne dispose d'aucune information sur eux. Le comportement de l'agent dépend de son environnement, qui inclut implicitement les autres, sans modélisation explicite de ces derniers. Dans ce contexte, des algorithmes classiques, tels que DQN, A2C ou PPO, peuvent être appliqués indépendamment à chaque agent du système.

Cependant, le MARL présente plusieurs difficultés : la non-stationnarité de l'environnement, l'absence de garantie de convergence vers un équilibre, des fonctions de récompense locales ou conflictuelles, ainsi que des problèmes de passage à l'échelle. Le paradigme CTDE permet d'atténuer certaines de ces difficultés. En effet, un entraînement centralisé permet de partager davantage d'informations entre les agents et de faciliter la coordination, ce qui améliore l'efficacité de l'apprentissage.

Une application du paradigme CTDE concerne les algorithmes actor-critic (AC). Le critic est utilisé uniquement durant l'entraînement, alors que l'actor est utilisé durant l'exécution. Ainsi, on peut introduire une information centralisée dans le critic sans compromettre l'exécution décentralisée. Le réseau critic peut donc prendre en entrée l'historique local combiné à une information globale (par exemple, l'état de tous les agents) afin d'estimer la fonction de valeur V . De la même manière, il est possible de construire un critic centralisé qui prend en compte les actions jointes, afin d'estimer la fonction action-valeur Q .

CONCLUSION

Le paradigme CTDE permet d'améliorer la stabilité et la coordination en MARL en utilisant l'information globale durant l'entraînement, tout en conservant une exécution décentralisée. Toutefois, il souffre de problèmes de scalabilité et de coûts computationnels élevés.

Son utilisation peut donc être pertinente dans le cas de notre étude, car elle permet d'améliorer l'apprentissage en utilisant des informations non disponibles lors de l'exécution. Cependant, le temps d'entraînement constitue une contrainte importante dans notre cas. Ainsi, cette approche est à envisager en fonction de la complexité du modèle et du compromis entre performance et coûts de calcul.

[16] Toward energy-efficient routing of multiple AGVs with multi-agent reinforcement learning, Ye & al. (2023)

A voir plus en profondeur si on veut s'intéresser à la problématique énergétique, pas pour le moment.